

Contents

SPECIFICATION — Specification Conformance Checker (speccheck; traceability graph, JUnit results, model-judged test strength; Python 3.12 + uv)	2
0. Intent and purpose	2
1. Actors and goals	3
2. Requirements (intent, high level)	4
3. Behavior and state model	8
3.1 Lifecycle	8
3.2 Executable flow	9
3.3 Durable artifacts	9
4. Interfaces / contracts	10
C-01 Spec ID grammar, declaration, citation, retirement	10
C-02 <code>SpecIndex</code>	12
C-03 <code>Citation</code> , <code>TestCase</code> , attribution	12
C-04 JUnit XML subset	14
C-05 Status algorithm and <code>IdStatus</code>	15
C-06 Judge interface	15
C-07 JSON report (<code>speccheck.json</code> , <code>schema_version</code> “1.3”)	17
C-08 Markdown report layout (<code>SPEC_CONFORMANCE_REPORT.md</code>)	19
C-09 LLM provider configuration	19
C-10 Judge instruction text (normative; F-004)	19
C-11 Judge progress indicator (R-30)	20
5. Interface specification	21
5.1 CLI (<code>speccheck</code>), the only surface	21
5.2 GUI	23
5.3 Diagnostics and verbosity contract	23
5.4 Exit codes	23
6. Invariants (must hold in every valid implementation)	23
7. Constraints (precise and measurable)	24
8. Edge cases and failure semantics	26
9. Acceptance criteria, tests, and evals	31
9.1 Extraction (C-01, C-02)	31
9.2 Citation and attribution (C-03)	32
9.3 Results mapping (C-04)	33
9.4 Status algorithm and metrics (C-05, C-07)	34
9.5 Judge contract (C-06)	35
9.6 Reports and determinism (C-07, C-08)	36
9.7 CLI, exit codes, diagnostics, boundary (§5)	37
9.8 Golden fixture (end to end)	40
9.9 Self-application (recorded, not gating)	41
9.10 Performance (recorded)	41
9.11 LLM judge evaluation (opt-in, <code>--judge llm</code> , not run in CI)	41
10. Dependencies and environment	42
11. Traceability matrix (id → where realized)	43
12. Open questions and decisions to confirm	46
Revision history	54

SPECIFICATION — Specification Conformance Checker (speccheck; traceability graph, JUnit results, model-judged test strength; Python 3.12 + uv)

- **Status:** v1.11 — v1.10 with all seven findings of the v1.10 `spec-review` applied (`SPEC_REVIEW_REPORT.md`, F-401..F-407): `schema_version` is stated once, in C-07, and every §9 row refers to it (F-401); the C-06 validation list is ordered and first-match, a non-string `clause` is absent, a coerced UNKNOWN records `clause ""` (F-402); the null-`verdict` condition names RECORDED ids in E-37, C-07 and T-34 (F-403); the `*(recorded)*` marker is one rule shared by both declaration forms and (b)’s title strips it, `~~T-48~~` (`recorded`) pinned (F-404); `judge_strength` excludes recorded ids, as a formula (F-405); K-15 trims the clause (F-406); editorial (F-407). v1.10 settled D-22 (recorded tests, R-35); v1.9 added clause-grounded verdicts (R-34) and a body in the judge-evaluation fixture (T-76). Earlier versions: see the revision history.
- **Language / stack:** Python 3.12 | standard library for the deterministic kernel (`re`, `ast`, `xml.etree`, `json`, `argparse`, `pathlib`) | CLI only; optional model-backed judge behind an `[llm]` extra
- **Sources:** `one_sentence_prompt.md` (the brief); `../skills/spec-writing/SKILL.md` (the ID taxonomy and `SPEC.md` shape the checker consumes); `../skills/spec-build/SKILL.md` §Phase 3 (the manual conformance audit this tool automates); `../skills/spec-review/SKILL.md` §3.17 (the intent → requirement → contract → invariant → test → evidence chain); `../outline.md` Chapters 15–18 (where this system is the worked example); `SPEC_REVIEW_REPORT.md` (one file, rewritten per review: v0.1 → F-001..F-017, v1.3 → F-201..F-210, v1.7 → F-301..F-307, v1.10 → F-401..F-407; all cited below); `FINAL_SPEC_REVIEW_REPORT.md` (review of v0.4; F-101..F-110 below point at it); `SPEC_v0.5_REVIEW_REPORT_by_QWEN.md` (independent review of v0.5 by a second model; its F-001..F-011 are cited below as Q-001..Q-011 to avoid collision); `PROPOSAL_v1.7_heading_bodies.md` (the v1.7 change, its evidence, and D-20); `PROPOSAL_v1.9_clause_grounding.md` (the v1.9 change, its two-model evidence, D-21, and the D-22 question)
- **Scope of this document:** The deterministic conformance kernel (spec-ID extraction, citation graph, test-result mapping, status computation, reporting) and the contract around the optional model-backed *judge*. It does not specify the quality of the specification under check (`spec-review` owns that), does not specify how tests are run (results are consumed, not produced), and does not specify any semantic analysis of source code.
- **Normative language:** MUST/MUST NOT/SHALL/SHALL NOT = normative; SHOULD = strong recommendation; MAY = optional.
- **Principle:** *The model may only ever make the news worse.* Every status is computed deterministically from evidence the operator can `grep`; the judge is permitted to downgrade a status with cited evidence, never to upgrade one, and every claim in the report points at a file and line.

0. Intent and purpose

`speccheck` answers one question about a project: **for every ID the specification declares, what evidence exists that the implementation realizes it?** It reads a `SPEC.md` written in the house ID taxonomy (R-nn requirements, C-nn contracts, I-nnn invariants, K-nn constraints, E-nn edge cases, T-nn tests), finds where each ID is cited in the source and test trees, joins those citations to the outcomes in a JUnit XML results file, and writes a conformance report in which every ID has exactly one status and every status has traceable evidence.

This is Phase 3 of `spec-build` (“re-read the spec and audit every artifact”) made mechanical. Humans

doing that audit are slow and get tired; agents doing it are fast and confabulate. The design splits the work accordingly:

- The **deterministic kernel** does everything that can be done by pattern, graph, and arithmetic: which IDs exist, which are cited where, which tests ran and how they ended, what that makes each ID’s status, and what the coverage ratios are. Two runs on identical inputs produce byte-identical reports.
- The **judge** — a model-backed component, off by default — answers exactly one question per (test case, ID) edge: *does this test assert the observable behavior this ID describes, or does it merely execute code near it?* — at the granularity of a clause: a statement with several clauses (a pinned interface and the prose around it, R-33) is asserted when a test asserts any one of them (C-10). Its answer MUST cite lines inside the test and MUST quote the clause of the statement it judged against; an answer without valid evidence, or whose clause cannot be found in the statement, is discarded as UNKNOWN (v1.9, R-34). The judge can turn a PASSING into a WEAKLY_PASSING; it can never turn anything into PASSING. It is never consulted about a *recorded* test — a T id marked **(recorded)** whose proof is a recorded run rather than an assertion (v1.10, R-35).

Why this boundary (context the implementer should not re-derive): a checker that lets a model decide conformance has merely moved the vibe-coding problem one level up — the report becomes something to trust rather than something to verify. Keeping the model on the downgrade-only side of the line means a green report is as trustworthy as `grep` plus the test runner, and a yellow one carries a reason you can click on.

Non-goals (explicit, to constrain the solution space):

- No semantic analysis of source code — no type inference, control-flow, or “does this function implement R-07” reasoning, by the kernel *or* the judge. Source citations are literal token matches.
- No test execution. The checker consumes a results file; it never runs `pytest`, `go test`, or anything else.
- No spec-quality review (ambiguity, contradictions, missing sections). That is **spec-review**.
- No remediation. The checker reports; it never edits the spec, the code, or the tests.
- No multi-repository or multi-spec runs; one spec, one source tree set, one results file per invocation.
- No IDE integration, daemon mode, watch mode, or web UI.
- Language adapters exist for Python (v0.1) and Swift (v1.6, R-31); every other language gets file-level attribution (O-2). No adapter parses its language with a real parser except Python (`ast`); the Swift adapter is line-based (D-17).
- The checker never reads its own outputs as inputs: the spec, the results file, and the two report files are excluded from every scan (C-03, F-002).

A known limitation, stated rather than hidden (F-013): citation is literal. A test that mentions [R-03] (#R-03) as *data* — asserting on a report that contains it, or on an error message — cites R-03 exactly as a test that proves it does. The `speccheck:ignore` markers in C-01 are the opt-out; they are the author’s responsibility, and `speccheck` never infers intent from context.

Relationship to the skills: `spec-writing` produces the input; `spec-review` grades it; `spec-build` builds from it and, in Phase 3, does by hand what this tool does by machine. The report this tool writes is intended to be pasted into `SPEC_BUILD_REPORT.md` as the per-ID evidence table.

1. Actors and goals

Actor	Goals
Operator (human at a terminal, or a CI job)	Run one command against a project and get a report plus an exit code that can gate a merge. Never has to trust a number without a path to its evidence.

Actor	Goals
Extractor (speccheck/extract.py)	Turn SPEC.md into the set of declared IDs (with title, statement text, family, retired flag) and turn source/test files into citations, deterministically.
Attributor (speccheck/attribute.py)	Map each citation in a test file to the test case that contains it (Python adapter), falling back to file-level attribution for anything it cannot parse.
Results Mapper (speccheck/results.py)	Read a JUnit XML file and join each <code><testcase></code> to an attributed test case, yielding outcomes per test case.
Grapher (speccheck/graph.py)	Build the ID $\rightarrow$ source-file / ID $\rightarrow$ test-case edge sets and compute each ID's deterministic status and the coverage metrics.
Judge (speccheck/judge.py; providers judge_mock.py, judge_llm.py)	For each (test case, ID) edge of a PASSING ID, return a verdict with evidence lines; the mock provider is deterministic, the LLM provider is opt-in. Single-principal: the judge sees one edge at a time and holds no state across calls.
Reporter (speccheck/report.py)	Render the Markdown and JSON reports from the graph and judge results, in a fixed order, with no content that varies between identical runs.
CLI (speccheck/cli.py, entry point speccheck)	The only surface. Parse arguments, wire the pipeline, apply the exit-code and verbosity contracts.

2. Requirements (intent, high level)

ID	Statement	Source
R-01	The checker MUST read a Markdown specification and extract every <i>declared</i> spec ID together with its family and statement text, per the grammar in C-01.	prompt; spec-writing §ID taxonomy
R-02	The checker MUST recognize a declared ID as <i>retired</i> when its declaration is struck through (<code>~~R-07~~</code>), and MUST exclude retired IDs from coverage denominators.	spec-writing §ID taxonomy (“retire old ones with a strike-through”)
R-03	The checker MUST scan every text file under each <code>--src</code> root, except the files C-03 excludes, and record a <i>source citation</i> for each (file, ID) pair where the ID token occurs, per C-03.	prompt (“analyzes an implementation”)
R-04	The checker MUST scan every text file under each <code>--tests</code> root, except the files C-03 excludes, and record a <i>test citation</i> for each (test case, ID) pair, attributing the citation to the enclosing test case when the file’s language adapter can delimit test cases, and to the file otherwise.	prompt (“and its tests”)
R-05	When <code>--results</code> is given, the checker MUST read a JUnit XML file and join each <code><testcase></code> to an attributed test case, recording the outcome passed , failed , error , or skipped per C-04.	spec-build Phase 1 exit gate

ID	Statement	Source
R-06	The checker MUST assign every declared, non-retired ID exactly one deterministic status from the set in C-05, computed by the algorithm in C-05 and by nothing else.	spec-build Phase 3.1
R-07	The checker MUST list every <i>dangling citation</i> : an ID token cited in a source or test file that is not declared in the specification.	spec-build Phase 3.1 (“no ID is dangling”)
R-08	The checker MUST list every <i>stale citation</i> : a citation of a retired ID.	R-02
R-09	The checker MUST compute the coverage metrics in C-07 §metrics, applying the zero-denominator rules stated there; it MUST NOT raise or emit 0.0 for an undefined ratio.	spec-writing §precision rules
R-10	When <code>--judge</code> is <code>mock</code> or <code>llm</code> , the checker MUST obtain, for each (test case, ID) edge whose ID is <code>PASSING</code> and whose test outcome is <code>passed</code> (F-104), one judge verdict from the set <code>ASSERTS</code> , <code>EXECUTES_ONLY</code> , <code>UNRELATED</code> , <code>UNKNOWN</code> , with evidence per C-06.	prompt (“AI-assisted”)
R-11	A judge verdict MUST only ever lower an ID’s status (<code>PASSING</code> → <code>WEAKLY_PASSING</code>); the judge MUST NOT change any status other than <code>PASSING</code> and MUST NOT create, remove, or re-attribute citations.	§0 principle
R-12	The checker MUST write a Markdown report (<code>SPEC_CONFORMANCE_REPORT.md</code>) laid out per C-08 in which every declared ID appears exactly once.	prompt (“traceable conformance report”)
R-13	The checker MUST write a JSON report (<code>speccheck.json</code>) conforming to C-07 that contains everything the Markdown report shows.	CI consumption
R-14	The checker MUST exit with the codes in §5.4, and the exit code MUST be a pure function of the JSON report content plus the <code>--strict</code> flag.	CI gating
R-15	With <code>--strict</code> , any in-scope ID whose status is not <code>PASSING</code> , any dangling citation, and any stale citation MUST cause exit 1. Status (C-05) is the only input to this rule; individual judge verdicts are not consulted again (F-003).	spec-build “done when”
R-16	With <code>--judge none</code> or <code>--judge mock</code> , two runs over byte-identical inputs MUST produce byte-identical <code>SPEC_CONFORMANCE_REPORT.md</code> and <code>speccheck.json</code> .	spec-writing §precision rules (determinism)
R-17	The checker MUST implement the diagnostics contract in §5.3: silent by default; <code>--verbose/--verbose INFO</code> emit metadata only; <code>--verbose DEBUG</code> additionally emits judge prompts and responses; all diagnostics go to <code>stderr</code> and never alter <code>stdout</code> or the report files.	spec-writing §5 cross-cutting contracts
R-18	With <code>--judge none</code> or <code>--judge mock</code> , the process MUST perform no network I/O, and <code>speccheck --self-check</code> MUST verify this boundary.	§0 boundary

ID	Statement	Source
R-19	The checker MUST treat all inputs as read-only; it MUST NOT create, modify, or delete any file outside <code>--out</code> , except as I-001 permits for <code>--self-check</code> (F-104).	safety
R-20	All paths in reports MUST be relative to <code>--root</code> (default: current directory) and use <code>/</code> as separator regardless of host OS.	R-16 (portability of golden reports)
R-21	On completion (exit 0 or 1) the checker MUST print exactly one summary line to stdout in the format of §5.1 and nothing else.	CI log hygiene
R-22	The mock judge MUST be deterministic and MUST NOT require configuration: it returns <code>ASSERTS</code> when the test case’s span contains an assertion token per C-06 , else <code>EXECUTES_ONLY</code> .	R-16 ; testability of the judge path
R-23	The LLM judge MUST read its endpoint, model name, and API key from environment variables per C-09 ; a missing variable MUST be a usage error (exit 2), and the key MUST never appear in any output, log, or report at any verbosity.	secret handling
R-24	Every deterministic status (C-05 steps 1–4), every count, and every metric in the report MUST be reproducible by an operator using only the report’s own evidence table, <code>grep</code> , and the results file; <code>WEAKLY_PASSING</code> and the judge-derived metrics MUST be reproducible from the same inputs plus the verdicts <i>as recorded</i> in the report (F-017).	prompt (“traceable”)
R-25	The checker MUST treat family T per C-05 : a T id’s status is determined by test citations and their outcomes only; source citations of a T id are recorded as evidence but never change its status, and a T id with no test citation is <code>UNCITED</code> , never <code>UNTESTED</code> (F-001).	spec-build (“every T-nn cites its spec ID”)
R-26	The LLM judge MUST use exactly the request body, response path, and instruction text pinned in C-06 and C-10 , and the report MUST record the SHA-256 of the instruction text used (F-004).	review F-004
R-27	The checker MUST honor the ignore markers in C-01 : a line containing <code>speccheck:ignore</code> yields no citations, and a file whose first three lines contain <code>speccheck:ignore-file</code> is not scanned; ignored files are counted in a Note (F-013).	review F-013
R-28	With <code>--strict</code> and <code>--judge llm</code> , the checker MUST exit 1 when the judge was unavailable (<code>judge_available == false</code>) or when <code>unknown_rate</code> exceeds <code>--max-unknown</code> (K-11), and the summary line MUST name the reason (F-012).	review F-012
R-29	The summary line MUST be a single line of ASCII text written to stdout as UTF-8 regardless of locale, in the exact format of §5.1 (F-010).	review F-010

ID	Statement	Source
R-30	With <code>--judge llm</code> , the checker MUST display a progress indicator for the judge stage on stderr, in the format of C-11, whenever <code>--progress</code> resolves to on (default <code>auto</code> : on iff stderr is a TTY and verbosity is not <code>DEBUG</code> ; §5.1). The indicator MUST be redrawn in place per K-13, MUST be erased before anything else is written to stderr or stdout after the judge stage begins, and MUST NOT alter stdout, either report file, or the exit code. With <code>--judge none</code> or <code>--judge mock</code> no indicator is ever drawn.	requester (2026-09-13: “speccheck should include progress bar when running with LLM judge, since that takes quite a bit of time”)
R-31	For every file ending <code>.swift</code> under a <code>--tests</code> root, the checker MUST delimit test cases per the C-03 Swift adapter — Swift Testing <code>@Test</code> functions and XCTest <code>test*</code> methods — with a span that begins at the first line of the doc comment / attribute block above the declaration and ends at the function’s closing brace, and MUST join their citations to SwiftPM’s xUnit <code><testcase></code> outcomes by function identifier (C-04), so that a citation written where <code>spec-build</code> puts it (the test’s doc comment) counts exactly as a Python citation does under R-04 and R-05.	requester (2026-09-17: “what would it take to make it work for Swift projects as well as Python ones?” — “go ahead”); the MonteCarloPi Swift build (SPEC_BUILD_REPORT.md of that project), where every Swift citation was file-level and the run reported 62 <code>unverified</code>
R-32	The declaration parser MUST declare an ID whose bold form is the <i>beginning</i> of a table row’s first cell and is followed by whitespace-separated decoration (<code>\ **[K-07] (#K-07)** **[port]** \ </code>), ignoring the decoration; the statement remains the second cell (C-01 (a), E-44).	the MonteCarloPi port spec marks port-specific ids <code>**[port]**</code> inside the id cell; six declared ids were reported as 43 dangling citations
R-33	For an ID declared by a heading (C-01 (b)), the checker MUST use the heading text followed by the section body beneath it — prose, bullets, and fenced code blocks, up to the next heading of the same or a higher level, capped per K-14 — as the ID’s statement (<code>SpecId.text</code> , <code>JudgeRequest.statement</code> , JSON <code>statement</code>), so that the judge and the JSON report see the contract’s pinned shape and not its title. The heading text alone is kept as <code>SpecId.title</code> / JSON <code>title</code> , and that is what the Markdown report renders (C-08). Table-row declarations are unchanged.	PROPOSAL_v1.7_heading_bodies.md (2026-09-17): two judges over the same 130 MonteCarloPi edges — <code>gpt-4o-mini</code> 65/70, 0 weak; <code>gemini-3.8-flash</code> 56/70, 9 weak, 15 UNRELATED, 3 UNKNOWN — and every disagreement was a <code>###</code> -declared contract whose statement was a title (<code>Data structures</code> , <code>EstimationWorker`</code> (an <code>actor`</code>)) while the pinned API sat unsent beneath it
R-34	For every <code>ASSERTS</code> or <code>EXECUTES_ONLY</code> verdict the judge MUST name the clause of the statement it judged against, as a verbatim excerpt, and the kernel MUST discard as UNKNOWN any such verdict whose excerpt cannot be located in the statement (K-15, E-48) — the statement-side mirror of the evidence-line rule (C-06, I-005), so that a recorded verdict is grounded in both the test and the statement.	PROPOSAL_v1.9_clause_grounding.md: with bodies in the statement, <code>gpt-4o-mini</code> graded long contracts by their gist — 13 of 18 stable downgrades on the mdv tree were tests asserting a body clause nearly verbatim — while a model that located the clause (<code>gemini-3.8-flash</code>) did not; nothing in the report showed <i>which</i> clause either had judged

ID	Statement	Source
R-35	A T id declared with the literal marker *(recorded)* immediately after its ID form (C-01) is RECORDED: the checker MUST compute its status by C-05 steps 1–4 exactly as for any other T id — it still needs a citing test case with a passed outcome to be PASSING — and MUST NOT send any of its edges to the judge, so that C-05 step 5 never applies to it; the marker MUST be carried into both reports (C-07 recorded, C-08). On a non-T id the marker is ignored with a Note (E-50).	D-22: the recorded tests of this document (T-48, T-49, T-51) are cited by presence checks so that they are not UNCITED; an honest judge reads such a check as EXECUTES_ONLY — gemini-3.8-flash did so to T-48 on 2026-09-18 — and --strict --judge llm went red for a reason the spec intended but had not written down

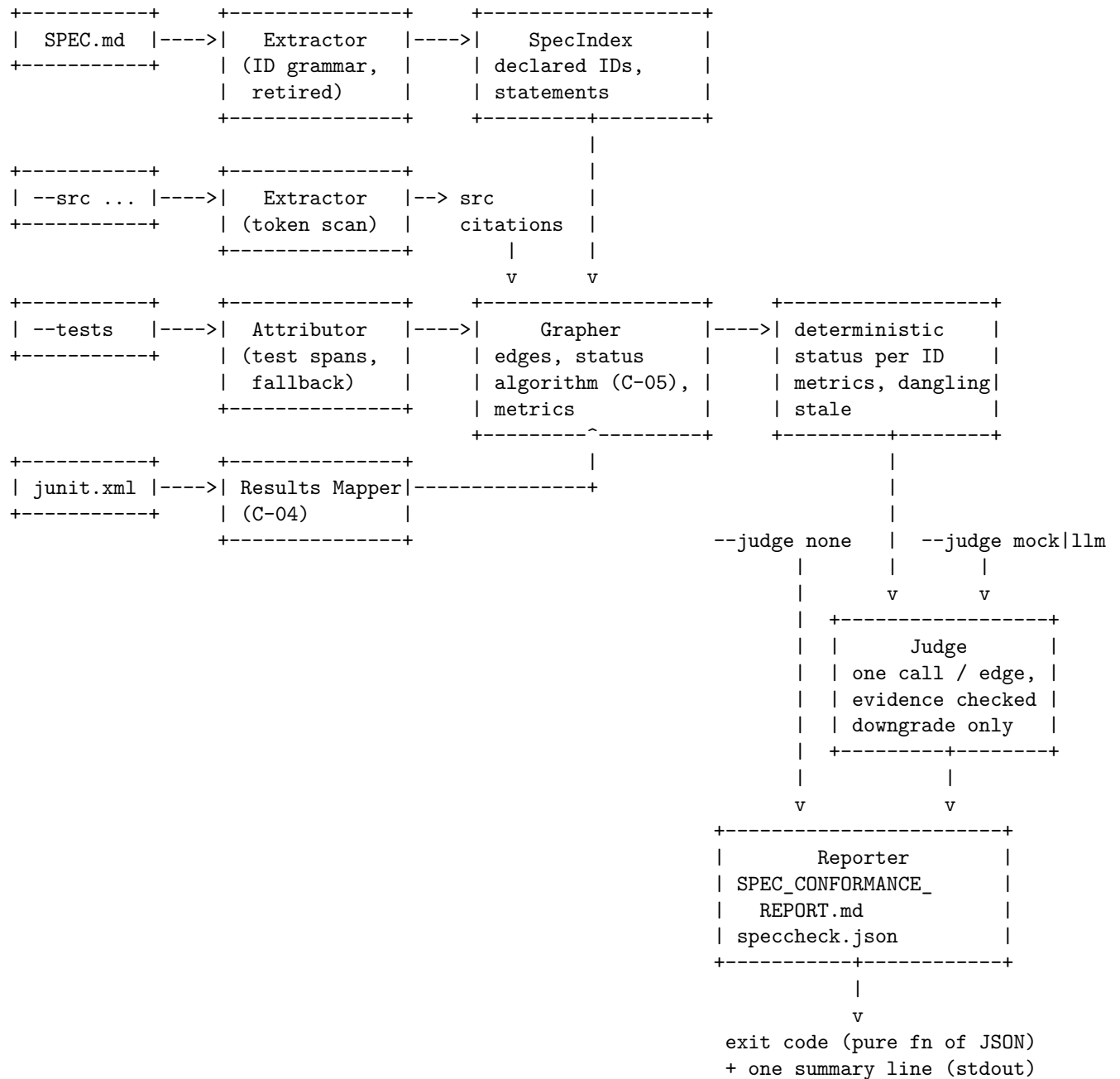
3. Behavior and state model

3.1 Lifecycle

A run is a single, stateless pipeline. There is no persistent state between runs, no cache, and no partial-output mode: either both report files are written or neither is. The Reporter guarantees this by rendering both documents in memory, then, in this order (F-106): (1) delete any leftover `<out>/ .speccheck.json.*.tmp` and `<out>/ .SPEC_CONFORMANCE_REPORT.md.*.tmp` from earlier runs; (2) write `.speccheck.json.<nonce>.tmp` and `.SPEC_CONFORMANCE_REPORT.md.<nonce>.tmp`, where `<nonce>` is 8 random hex digits chosen per run (F-103); (3) rename the JSON temporary to `speccheck.json`; (4) rename the Markdown temporary to `SPEC_CONFORMANCE_REPORT.md`. If any step fails — an interrupt (E-41) included — every temporary and any file this run already renamed is removed before exit 3 (F-015). The nonce never appears in either report, so I-002 is unaffected.

Stage	Owner	Input	Output	Fails with
parse-args	CLI	argv, env	Config	exit 2
extract-spec	Extractor	SPEC.md	SpecIndex (C-02)	exit 3 (E-01, E-02, E-03)
scan-src	Extractor	--src roots minus C-03 exclusions	Citation[] (C-03)	never (E-10, E-11 are notes)
scan-tests + attribute	Extractor, Attributor	--tests roots	TestCase[], Citation[]	never (E-12 is a note)
map-results	Results Mapper	--results	Outcome[] (C-04)	exit 3 (E-05)
graph + status	Grapher	all of the above	IdRecord[] with deterministic status; dangling/stale lists; metrics	never
judge (optional)	Judge	PASSING edges	Verdict[] (C-06); progress indicator on stderr while running (R-30, C-11)	never (E-14..E-17 yield UNKNOWN)
report	Reporter	everything	SPEC_CONFORMANCE_REPORT.md, out unwritable speccheck.json	—
exit	CLI	JSON report, --strict	exit code (§5.4), summary line	—

3.2 Executable flow



3.3 Durable artifacts

Artifact	Written to	Version	Shape
SPEC_CONFORMANCE_REPORT.md	stdout (default .)	mirrors JSON schema_version	C-08

Artifact	Written to	Version	Shape
speccheck.json	--out	schema_version: "1.3" ("1.0" through v1.6; "1.1" added title in v1.7; "1.2" clause per verdict in v1.9; "1.3" recorded per id in v1.10)	C-07

Both files are overwritten on every successful run, via the temp-and-rename sequence in §3.1. No other file is ever written, and no temporary survives a run (I-001).

4. Interfaces / contracts

C-01 Spec ID grammar, declaration, citation, retirement

```
ID      := FAMILY "-" DIGITS
FAMILY  := "R" | "C" | "I" | "K" | "E" | "T"          # 0-n, D-nn and F-nnn are NOT conformance IDs
DIGITS  := [0-9]{1,3}
TOKEN   := ID bounded by non-alphanumerics on both sides # regex: (?![A-Za-z0-9])([RCIKET])-( [0-9]
```

Declaration (in SPEC.md only) - an ID is DECLARED when, outside fenced code blocks, either form occurs (F-009). A fenced code block (F-108) opens at a line whose first non-space characters are three backticks (U+0060 x3) or three tildes (~~~), optionally followed by an info string, and closes at the next line whose first non-space characters are the same three-character marker - the same character as the opener - followed by nothing but whitespace; a closing line's info string is not permitted, a backtick fence is never closed by tildes nor a tilde fence by backticks, and an unclosed fence runs to end of file (Q-008).

Lines (F-301): SPEC.md is split into lines on "\n"; a trailing "\r" on any line is removed before every other rule is applied, so a CRLF file parses as its LF twin and yields byte-identical output (I-002). A BLANK line is one whose text is empty or consists only of spaces and tabs. Body lines (see (b)) are re-joined with "\n"; no other whitespace is added or removed - trailing spaces on a body line are kept.

(a) table row: | ****R-07**** | <statement> | ...
row := a line whose first non-space character is "|"
cells := the row split on every "|" that is not preceded by "\" and not inside a
 backtick span (`...`); the leading and trailing empty cells are dropped
the trimmed content of the FIRST cell MUST BEGIN with one of the forms
****ID**** **~~~~ID***** *****~ID*****
and whatever follows the form MUST be empty or begin with whitespace (R-32, E-44):
`**\*\*K-07\*\*** **\*\*[port]\*\***` declares K-07; `**\*\*K-07\*\*x`** and `**\*\*K-07\*\***`, note` declare nothing.
Text after the form is DECORATION: it is not part of the statement, and a bold ID inside
it is not a declaration (`**\*\*R-01\*\*** **\*\*R-02\*\***` declares R-01 only).
RECORDED marker - see the rule after (b); in a row it is the first token of the DECORATION:
`| **\*\*T-48\*\*** **\*(recorded)\*** | ... |` and `| **~~~~T-48\*\*\*** **\*(recorded)\*** | ... |` (R-35, F-404)
statement := the trimmed text of the SECOND cell ("" if there is none)
title := statement (a table declaration has no separate title; R-33)
A bold ID in any other cell is not a declaration (E-31). A separator row (cells made of
"-" and ":" only) is never a declaration.

(b) heading: **### C-03** <title>
HEADING LINE := a line, outside fenced code blocks, indented by at most three spaces, whose
first non-space characters are one to six "#" followed by whitespace or by end

of line; its LEVEL is the number of "#". ATX headings only (F-304): a line indented four or more spaces is not a heading (and not a declaration), a line underlined by === or --- (setext) is not a heading, "###C-03" (no space) is not a heading, and a bare "####" is a HEADING LINE with an empty title.

the declaring line MUST be a HEADING LINE and the ID token (optionally wrapped ~ ~) MUST be the first token after the "#" run; "### 9.1 Extraction (C-01, C-02)" declares nothing.

title := the rest of the heading text after the ID token - and after the RECORDED marker when the token that follows the ID token is exactly `\*(recorded)\*` (rule below) - trimmed, whitespace-collapsed, with a trailing run of "#" that is preceded by whitespace removed ("### C-03 Title ###" -> "Title"; "### T-48 *(recorded)* Run" -> "Run")

statement := title, then - when the SECTION BODY is non-empty - a newline and the SECTION BODY; when title is "" and the body is non-empty, the SECTION BODY alone (F-305). (R-33; v1.7. Through v1.6 the statement was the title alone.)

SECTION BODY := the lines after the HEADING LINE up to (not including) the next HEADING LINE whose LEVEL is <= the declaring heading's LEVEL, or end of file; fenced code blocks INCLUDED (the pinned shape is the contract; D-20); leading and trailing blank lines dropped; whitespace inside preserved line for line (a code block's indentation is meaning); the body's own deeper headings (#### under a ###) are part of it; a thematic break (---) is body text like any other line; capped at K-14 bytes (E-46). A body that is empty after trimming - "### C-04 Data structures" followed by blank lines and another ### - leaves the statement equal to the title (E-46).

The body is context, not grammar: a table row inside it is still parsed for its own declarations, a deeper heading inside it still declares its own ID (whose own body is delimited by the same rule), and a bold ID in the body is not a declaration of the heading's ID (E-31, E-47). IDs inside the body's fenced blocks remain non-declarations (E-04).

RECORDED marker (R-35, v1.10; one rule for both forms, F-404): a declaration is RECORDED when the whitespace-separated token immediately after its ID form is exactly `\*(recorded)\*` - in a row, the first token of the DECORATION; in a heading, the first token after the ID token, which is then not part of the title. The marker is meaningful for family T only; on any other family it is ignored and noted (E-50). `\*(Recorded)\*`, `(recorded)`, and a marker that is not that first token are decoration like any other and mark nothing. A retired declaration may also be recorded. A SECOND declaration of the same ID is E-02 (exit 3); the message names the ID and both lines. Nothing "wins": the run produces no report (Q-006).

Retirement - a declaration whose ID token is wrapped in ~ ~ ~ :

| ~~~R-07***~ | ... | or | ***~R-07***~ | ... or ### ~C-03~ ...

A retired ID is DECLARED and RETIRED. Retirement is not un-done by a later plain declaration (E-03).

Citation (in --src / --tests files) - every TOKEN occurrence in a text file is a citation, regardless of comments, strings, or code. Fenced code blocks are NOT excluded in source files (only in SPEC.md). ID numbers are compared numerically: R-7 and R-07 and R-007 are the same ID.

Ignore markers (F-013), matched literally and case-sensitively:

speccheck:ignore anywhere on a line -> that line yields no citations (the line still counts toward a test case's span)

speccheck:ignore-file anywhere in the first 3 lines of a file -> the file is not scanned at all (no citations, no test cases); one Note per run: "ignored N file(s) by speccheck:ignore-file"

The markers have no effect inside SPEC.md.

Normalized form used everywhere in output: FAMILY "-" zero-padded to 2 digits for R/C/K/E/T, 3 digits for I (R-07, I-005). An ID declared as R-7 is reported as R-07.

Family T (F-001): a T id names an acceptance test. It is IN SCOPE and is expected to be cited by the test case that realizes it (in that case's docstring or a comment). Source citations of a T id are recorded but do not affect its status (C-05 step 2b). All six families count toward ``in_scope`` and ``conformance``. A RECORDED T id (marker above; R-35) is a test whose proof is a recorded run (a self-application, an evaluation, a benchmark) rather than an assertion: it is still cited by a test case - typically a presence check that the recorded artefact exists and has the right shape - and that citation and its outcome give the id its status; its edges are never judged (C-05 step 5, I-010).

C-02 SpecIndex

```
@dataclass(frozen=True)
class SpecId:
    family: str          # one of "R","C","I","K","E","T"
    number: int          # 0..999
    title: str           # table: the second cell, whitespace-collapsed, may be ""; heading: the heading
                        # remainder, whitespace-collapsed (C-01). What C-08 renders. Added in v1.7 (R-
    text: str            # the statement (C-01). table: == title. heading: title + "\n" + SECTION BODY
                        # when the body is non-empty (body lines per the C-01 Lines rule: "\r" stripped
                        # re-joined with "\n", other whitespace preserved; fenced blocks included; cap
                        # per K-14); == title when the body is empty; == body when title is "" (R-33,
    line: int            # 1-based line of the declaration in SPEC.md
    retired: bool
    recorded: bool       # C-01 RECORDED marker (R-35); only ever True for family T

@dataclass(frozen=True)
class SpecIndex:
    path: str            # relative, POSIX
    ids: tuple[SpecId, ...] # sorted by (family order R,C,I,K,E,T ; number)
    # invariant: no two SpecId share (family, number)
```

C-03 Citation, TestCase, attribution

```
@dataclass(frozen=True)
class TestCase:
    file: str            # relative, POSIX
    name: str            # function name, e.g. "test_zero_rate"; "" for file-level
    classname: str       # dotted module path used to match JUnit classname, e.g. "tests.test_core"
    start: int           # 1-based first line of the span; 1 for file-level
    end: int             # inclusive last line; last line of file for file-level

@dataclass(frozen=True)
class Citation:
    id: str              # normalized ID
    file: str
    line: int
    kind: str            # "src" | "test"
    testcase: TestCase | None # for kind=="test": the enclosing case, or the file-level case
```

Python adapter (files ending .py under a --tests root):

- * parse with ``ast``; a test case is (F-007):
 - every top-level FunctionDef or AsyncFunctionDef whose name starts with `"test_"`;

- every FunctionDef/AsyncFunctionDef whose name starts with "test" (underscore NOT required: "testAddition" and "test_addition" both qualify; F-102) that is a direct member of a top-level ClassDef which is RECOGNIZED: its name starts with "Test", OR any of its bases is a Name/Attribute whose final identifier ends with "TestCase";

span = node.lineno .. node.end_lineno (decorators included, per `ast`).

- * a "test*" function that is a member of an UNRECOGNIZED top-level class, or of a nested class, is not a test case; its lines belong to the file-level case, and the file gets one Note: "undelimited tests in <path>: <Class.name>, ..." (E-28).
- * classname = the file path relative to --root with "/" -> "." and ".py" dropped ("tests/test_core.py" -> "tests.test_core"); methods append "." + ClassName.
- * a citation on a line inside a span is attributed to that case; any other citation in the file is attributed to the file-level case (name "").
- * a .py file that fails to parse -> whole file is file-level (E-12; a note, not an error).

Swift adapter (files ending .swift under a --tests root; R-31, D-17, D-18, D-19). The file is delimited BY LINES, not by a parser, so the kernel stays standard-library (D-17). Every rule below reads a line's text with its `//` comment removed (a `//` inside a string literal is treated as a comment start too - D-17) and counts braces `{` / `}` only outside `"...` and `"...` string literals (the latter may span lines).

TYPE LINE := a line matching, after that stripping,

```

\s*(?:@S\s+|(?:(public|package|internal|private|fileprivate|open|final|indirect)\s+
(?:struct|class|actor|enum|extension)\s+([A-Za-z_][A-Za-z0-9_]*))

```

whose brace depth is greater at the end of the line than at its start. Its CHAIN is the identifiers of the TYPE LINES still open at that depth (outermost first) plus its own identifier; an `extension X` line contributes `X`. Nesting depth is unbounded.

FUNC LINE := a line matching, after that stripping,

```

\s*(?:@[A-Za-z_][A-Za-z0-9_]*(?:\([^\)]*\))?\s+|(?:(public|package|internal|private|
func\s+([A-Za-z_][A-Za-z0-9_]*))\s*(<[

```

whose identifier is the FUNC NAME. A FUNC LINE on which no `{` opens before SPAN END can be computed (a protocol requirement, a declaration without a body) is not a case.

ATTRIBUTE BLOCK := the maximal run of lines immediately above the FUNC LINE, each of which is

- (i) a doc-comment line: first non-space characters `///`, or any line of a `/\*\* ... \*/` block; or
- (ii) an attribute line: first non-space character `@`, or a continuation of an attribute whose parentheses had not balanced on the previous line of the run; or
- (iii) a blank line that lies between two lines of kinds (i)/(ii);

together with the text of the FUNC LINE itself that precedes `func`.

SPAN END := the first line, at or after the FUNC LINE, on which the running brace depth returns to the depth in force before the FUNC LINE's first `{`.

A test case is:

- (S) Swift Testing - a FUNC LINE whose ATTRIBUTE BLOCK contains the token `@Test` bounded on the right by `(`, whitespace or end of line (`@Testable`, `@TestSuite` do not match).
name := FUNC NAME (the parameter list is not part of the name; D-19)
classname := MODULE, or MODULE "." + CHAIN of the innermost enclosing TYPE LINE joined with "."
- (X) XCTest - a FUNC LINE whose FUNC NAME starts with `test`, that is a DIRECT member (brace depth exactly one deeper) of a TYPE LINE declaring a `class` whose text contains the token `XCTestCase` after the identifier.
name := FUNC NAME; classname := MODULE "." + that class's CHAIN joined with "."

A FUNC LINE that satisfies both is (S). A FUNC LINE whose FUNC NAME starts with `test` and is neither (S) nor (X) - a method of a non-`XCTestCase` type, a member of a class nested inside an `XCTestCase`, a free function - is NOT a test case: its citations are file-level and the file gets one Note "undelimited tests in <path>: <Chain.name>, ..." (E-43; E-28's rule). Any other function is a helper and produces no Note.

MODULE := the first path component of the file's path relative to the --tests root under which it was found, or the last path component of that root when the file lies directly under it

(SwiftPM's `Tests/<Target>/...` layout; D-18). `Tests/ProbeTests/Unit/Foo.swift` under `--tests Tests` -> `ProbeTests`. (Under `--tests Tests/ProbeTests` the same file yields `Unit`, which will **not** join; **pass** the `Tests` directory.)

span := first line of the ATTRIBUTE BLOCK .. SPAN END. A citation on a line inside a span **is** attributed to that case (the doc comment **is** inside the span - that **is** the point of R-31); every other citation **is** file-level (E-13).

A .swift file whose brace depth goes negative on **any** line, **or is** non-zero at end of file, **is** one file-level case with Note `parse fallback: <path>` (E-42; E-12's rule).

What the join sees (measured 2026-09-17, Swift 6.4 / SwiftPM; D-19): `swift test --xunit-output junit.xml` writes `junit-swift-testing.xml` for Swift Testing with

```

    classname = "<Module>.<Outer>.<Inner>" (dotted CHAIN; a free function: "<Module>")
    name      = "<name>(<label>:<label>:)" (the signature; ONE <testcase> per function -
                                           parameterized tests are not expanded; a
                                           `.disabled` test carries <skipped>)

```

and, on toolchains that write it, `junit.xml` for XCTest with classname = "<Module>.<Class>" and name = "<testName>". C-04 strips the signature, so both forms join by identifier.

Fallback adapter (**any** other text file): one file-level case per file; its classname **is** derived exactly **as for** Python (path relative to --root, "/" -> ".", final extension dropped; F-108).

Text vs binary (F-008): a file **is** BINARY if **any** of its first 8192 bytes **is** 0x00. Binary files are skipped silently (no Note, no citations, no test case). Everything **else is** text **and is** decoded **as** UTF-8 with errors="replace" (E-11). Symbolic links - to files **or** to directories - ENCOUNTERED DURING DESCENT are never followed (K-03); one Note per run if **any** were skipped (E-30). A path given on the command line (--spec, --src, --tests, --results, --out, --root) **is** resolved once, checked against --root (E-09), **and** then used **in** its resolved form: a symlinked --src root **is** descended, **and it is not** counted **as** a skipped symlink (Q-007).

Excluded **from** every scan (F-002), whether **or not** they lie under a --src/--tests root, **and** whether **or not** they exist yet: the file named by --spec, the file named by --results, <out>/SPEC_CONFORMANCE_REPORT.md, <out>/speccheck.json, **and** every file directly under <out> whose name starts with ".speccheck.json." or ".SPEC_CONFORMANCE_REPORT.md." **and** ends with ".tmp" (the §3.1 temporaries, including leftovers **from** a killed run; F-103). Exclusion **is** by resolved path (symlinks resolved), **is** silent (no Note), **and** applies before K-02/K-03 filtering.

C-04 JUnit XML subset

Root: <testsuites> or <testsuite>. Every <testcase> element anywhere below the root is an outcome.

```

    classname := @classname (may be ""; an empty classname ends every dotted name on a component
                           boundary, so it matches every TestCase with the same join_name, and the
                           longest-suffix rule below then applies with suffix length 0 - F-105)
    name      := @name (required; missing -> E-05)
    outcome   := "failed"  if a <failure> child exists
               | "error"   if an <error> child exists
               | "skipped" if a <skipped> child exists
               | "passed"  otherwise

```

Join to a TestCase (F-006): a result joins the TestCase whose `join\_name` equals the result's join_name AND whose dotted `classname` ends with the result's dotted classname on a component boundary ("tests.test_core" matches "tests.test_core" and "test_core"; it does not match "unit_test_core"). Among candidates, the one whose classname has the LONGEST common dotted suffix with the result wins; if two or more candidates tie, the result is UNATTRIBUTED and one Note names the result and every candidate (E-27). `join\_name` is defined as:

```

    join_name := step 1: if name ends with "]" and contains "[": name[:name.index("[")]
                   else:
                       name
                   i.e. everything from the FIRST "[" to the end is removed, whatever it contains

```

```

(F-005, F-101): "test_x[3-True]" -> "test_x"; "test_x[list[int]]" -> "test_x";
"test_y[a][b]" -> "test_y".
step 2 (R-31, D-19): if the step-1 result ends with ")" and contains "(":
everything from the FIRST "(" is removed as well - a Swift signature names the
test, not a variant: "twoArgs(a:b:)" -> "twoArgs"; "freeFunction()" ->
"freeFunction"; "testAddition" -> "testAddition"; "test_x[f(1)]" -> "test_x".
param      := the text removed in step 1 without its outer brackets ("3-True", "list[int]",
"a)[b]", or null; a signature removed in step 2 is never a param.
Two Swift Testing cases in one type with the same identifier (overloads by label: `f(a:)` and
`f(b:)` share `join_name` and `classname`: a result for either ties, is UNATTRIBUTED, and one
Note names the result and both candidates (E-45; the E-27 rule).
Every <testcase> that joins the same TestCase (parametrized variants, or duplicates) is kept in
that case's `results` list; the case's single `outcome` = worst of them, worst order
error > failed > skipped > passed (E-06, E-24).
A <testcase> that joins no TestCase is an UNATTRIBUTED result (E-07): listed, never counted.
Malformed XML, or a root that is neither element: E-05 -> exit 3.

```

C-05 Status algorithm and IdStatus

```

IdStatus (deterministic):  RETIRED | UNCITED | UNTESTED | UNVERIFIED | FAILING | SKIPPED | PASSING
IdStatus (judge-only):     WEAKLY_PASSING

```

For each declared ID x:

1. if x.retired -> RETIRED (excluded from denominators)
2. T := { test cases with >=1 test citation of x } # file-level cases included
S := { source files with >=1 src citation of x }
2a. if x.family != "T":
if T == {} and S == {} -> UNCITED
if T == {} -> UNTESTED
2b. if x.family == "T": # F-001: S is evidence only
if T == {} -> UNCITED
3. if no --results given, or no case in T has an outcome -> UNVERIFIED
4. O := outcomes of cases in T that have one
if any o in O is failed or error -> FAILING
elif every o in O is skipped -> SKIPPED
else -> PASSING
5. (judge enabled only) if status == PASSING and not x.recorded: # R-35: recorded ids skip step
V := verdicts for edges (t, x), t in T with outcome passed
if V is non-empty and no v in V is ASSERTS and some v in V is EXECUTES_ONLY or UNRELATED
-> WEAKLY_PASSING
else -> PASSING (UNKNOWN never changes status)

Cases in T without an outcome are reported as "unrun" for x and ignored in step 4.

The judge is never consulted for edges whose ID is not PASSING after step 4, nor for any edge of a RECORDED id (I-010, R-35); a recorded id's `tests[].verdict` is null (E-37) and its edges are not eligible edges for the progress indicator's <total> (C-11).

Step 5 is the ONLY place verdicts influence anything: an ID with verdicts {ASSERTS, EXECUTES_ONLY} is PASSING, and PASSING is what --strict tests (R-15, §5.4; F-003). There is no per-edge rule anywhere else in this document.

C-06 Judge interface

```

@dataclass(frozen=True)
class JudgeRequest:
    id: str # "R-07"

```

```

statement: str          # SpecId.text - for a heading-declared ID: title, newline, section body (R-
testcase: TestCase
source: str             # lines testcase.start..testcase.end, each prefixed "<lineno>\t" (F-109)

@dataclass(frozen=True)
class Evidence:
    file: str
    line: int            # MUST satisfy testcase.start <= line <= testcase.end

@dataclass(frozen=True)
class Verdict:
    verdict: str         # "ASSERTS" / "EXECUTES_ONLY" / "UNRELATED" / "UNKNOWN"
    clause: str          # verbatim excerpt of the statement judged against (K-15); "" for UNRELATED
    evidence: tuple[Evidence, ...]
    rationale: str       # <= 280 characters, single line

class Judge(Protocol):
    def judge(self, req: JudgeRequest) -> Verdict: ...

```

Validation applied by the kernel to EVERY provider's raw answer (judge.py, not the provider).

The rules are applied IN THIS ORDER; the first rule that fires determines the verdict and the rationale, and no later rule is evaluated (F-402). Rule 8 is applied to whatever rationale results. A verdict coerced to UNKNOWN by any rule is recorded with `clause` "" (E-49).

1. provider raised, timed out (K-05), or returned non-JSON -> UNKNOWN, rationale = "judge: <class of error>"
2. verdict not in the four-value set -> UNKNOWN, rationale = "judge: malformed verdict"
3. clause absent, or present but not a JSON string -> clause := "" (then continue)
4. verdict is UNRELATED or UNKNOWN -> clause := "" whatever was returned
5. verdict is ASSERTS or EXECUTES_ONLY and clause is not LOCATED (K-15) -> UNKNOWN, rationale = "judge: unlocated"
6. verdict == ASSERTS and evidence is empty -> UNKNOWN, rationale = "judge: ungrounded"
7. any evidence.file != testcase.file, or line outside span -> UNKNOWN, rationale = "judge: ungrounded"
8. rationale longer than 280 chars -> truncated to 277 + "..."

An UNKNOWN produced by validation is recorded with `coerced: true` in the JSON report.

LLM provider wire format (judge_llm.py; F-004) - an OpenAI-compatible chat-completions endpoint:

```

Request: POST SPECHECK_JUDGE_URL
Headers: Content-Type: application/json ; Authorization: Bearer <SPECHECK_JUDGE_API_KEY>
Body (JSON, exactly these keys):
{ "model": <SPECHECK_JUDGE_MODEL>,
  "temperature": 0,
  "max_tokens": 4000,
  "messages": [
    { "role": "system", "content": <the C-10 instruction text, verbatim> },
    { "role": "user", "content": <JSON object {id, statement, file, start, end, source}
                                statement = SpecId.text verbatim - body, fenced block
                                indentation included for a heading-declared ID (R-3)
                                source = the span with every line prefixed by its
                                1-based line number and one TAB (F-109)> } ] }

```

Response: HTTP 200 with a JSON body; the model's text is read from choices[0].message.content .

Any other status, a non-JSON body, or a missing path -> E-14 / E-15.

The text is stripped of surrounding whitespace and of ONE enclosing ``` or ```json fence if present, then MUST parse as a single JSON object {verdict, clause, evidence:[{file,line}, {rationale}], `clause` MAY be absent (it is then ""); anything else is non-JSON (-> UNKNOWN, I-10).

Exactly one HTTP request per edge; no retries (K-06). The SHA-256 of the C-10 text as sent is

recorded in the report as `judge\_prompt\_sha256` (C-07).

Mock provider (judge_mock.py):

```
verdict = ASSERTS if the span contains an assertion token, else EXECUTES_ONLY
assertion token := a line matching ^\s*assert\b or containing .assert or pytest.raises(
    or (Swift; R-31) containing #expect( or #require( or XCTAssert or XCTFail(
evidence = every such line (file, line); rationale = "mock: assertion token on N line(s)" / "mock: no
clause = the whitespace-collapsed statement cut to its first 280 characters - a prefix, so it is al
under K-15 (equal to a statement shorter than 12 characters, including ""); the mock reads
semantics and stays deterministic (R-16, R-22)
```

C-07 JSON report (speccheck.json, schema_version “1.3”)

```
{
  "schema_version": "1.3",
  "spec": "SPEC.md",
  "judge": "none | mock | llm",
  "judge_available": null,
  "judge_prompt_sha256": "<64 hex chars; present only when judge == llm>",
  "strict": false,
  "max_unknown": 0.2000,
  "strict_judge_failure": null,
  "ids": [
    {
      "id": "R-07",
      "family": "R",
      "recorded": false,
      "title": "...",
      "statement": "...",
      "line": 88,
      "status": "PASSING",
      "src": [ {"file": "src/pkg/core.py", "lines": [12, 40]} ],
      "tests": [
        {
          "file": "tests/test_core.py", "name": "test_zero_rate", "classname": "tests.test_core",
          "lines": [17], "outcome": "passed",
          "results": [ {"name": "test_zero_rate[0]", "param": "0", "outcome": "passed"},
                      {"name": "test_zero_rate[1]", "param": "1", "outcome": "passed"} ],
          "verdict": {"verdict": "ASSERTS", "clause": "returns the arithmetic sum of `a` and `b`",
                     "evidence": [{"file": "tests/test_core.py", "line": 21}],
                     "rationale": "...", "coerced": false} // null when the case was not judged (Q-001)
        }
      ],
      "unrun": [ {"file": "...", "name": "..."} ]
    }
  ],
  "dangling": [ {"id": "R-99", "file": "src/pkg/x.py", "line": 3} ],
  "stale": [ {"id": "R-02", "file": "tests/test_old.py", "line": 9} ],
  "unattributed_results": [ {"classname": "tests.test_gone", "name": "test_x", "outcome": "passed"} ],
  "notes": [ "skipped 1 file over 2 MiB: data/big.json", "parse fallback: tests/test_bad.py" ],
  "metrics": {
    "declared": 61, "retired": 2, "in_scope": 59,
    "by_status": {"PASSING": 50, "WEAKLY_PASSING": 3, "FAILING": 1, "SKIPPED": 0, "UNVERIFIED": 2, "UNTESTED": 1},
    "conformance_ratio": "50/59", "conformance": 0.8475,
  }
}
```

```

    "by_family": { "R": {"in_scope": 24, "passing": 22, "ratio": 0.9167}, "C": {"...": "..."}, "I": {"in_scope": 24, "passing": 22, "ratio": 0.9167}, "judge_strength_ratio": "50/53", "judge_strength": 0.9434, "unknown_rate": 0.0000
  },
  "exit_code": 1
}

```

Rules: keys emitted in exactly this order (``judge_prompt_sha256`` omitted entirely unless `judge == llm`; ``max_unknown`` always present, echoing K-11; ``strict_judge_failure`` is null, "unavailable", or "unknown_rate" and is non-null only when R-28 forced exit 1 - when BOTH reasons hold, "unavailable" is recorded, since a fully unavailable judge makes the unknown rate vacuous; Q-004); ``ids`` sorted by (family order R,C,I,K,E,T ; number); ``results`` sorted by name; ``src`/`tests`/`dangling`/`stale`/`unattributed_results`` sorted by (file, line) then name; ``notes`` is a list of strings sorted ascending by Unicode code point (each Note's text is fixed by the E-case that produces it, so this order is total and reproducible; Q-002); ``lines`` ascending, unique. No timestamps, hostnames, absolute paths, durations, or version strings other than `schema_version` (R-16).

Title and statement (R-33): ``title`` is ``SpecId.title`` and ``statement`` is ``SpecId.text`` (C-01, C-02). For a table-declared ID the two are equal; for a heading-declared ID with a body, ``statement`` is multi-line - title, ``\n``, the section body with its fenced blocks and indentation, truncated per K-14 when over the cap - and ``title`` is the heading text alone. ``schema_version`` was ``"1.0"`` through v1.6, ``"1.1"`` from v1.7 (``title`` added), and ``"1.2"`` from v1.9 (``clause`` added to every verdict object, after ``verdict``; R-34), and ``"1.3"`` from v1.10 (``recorded`` added to every id record, after ``family``; R-35 - ``true`` only for a T id carrying the C-01 marker); no other key changed. The literal is stated in this contract (heading and example) and in §3.3 only; no §9 row repeats it - a test asserts "the C-07 value" (F-401).

Verdict field (Q-001): every ``tests[]`` entry has the key ``verdict``. It is a verdict object when the case was judged (judge enabled, ID ``PASSING`` after C-05 step 4, ID not RECORDED, outcome ``passed`` - the E-37 condition) and ``null`` otherwise - including every entry under ``--judge none``. The key is never omitted.

Names (Q-011): ``tests[].name`` is the ``TestCase.name`` - ``""`` for a file-level case. ``(file)`` is a Markdown-only rendering (C-08) and never appears in the JSON. R-13's "contains everything" concerns data; a rendering label is not data.

Numbers (Q-009): every ratio is computed as `Decimal(numerator) / Decimal(denominator)` under a 28-digit context and quantized to 4 places with `ROUND_HALF_EVEN`; it is emitted as a JSON number with exactly four decimal places (0.9000, not 0.9), so the JSON text is byte-determined without reference to binary floating point. K-11 compares these quantized values. ``<pct>`` in the summary line is the quantized ``conformance`` times 100, quantized to 1 place with `ROUND_HALF_EVEN`. ``max_unknown`` is not a ratio but is emitted the same way: the K-11 Decimal quantized to 4 places (0.2000), matching the summary-line suffix (F-206).

Metrics:

```

conformance          = |PASSING| / in_scope                                (WEAKLY_PASSING is NOT passing)
by_family[f].ratio    = |PASSING in f| / |in_scope in f|
judge_strength        = |PASSING \ RECORDED| / (|PASSING \ RECORDED| + |WEAKLY_PASSING|)  (judge enabled)
                                                                RECORDED ids are PASSING without a judged edge and are outside this population -
                                                                `conformance` and `by_family` DO include them, since they are PASSING by citation
unknown_rate          = |edges with verdict UNKNOWN| / |judged edges| (judge enabled only; else absent)
Zero denominators: ratio = null (JSON) / "n/a" (Markdown); the "a/b" string is still emitted ("0/0");
`judge_strength` is null when every PASSING id is RECORDED and nothing is WEAKLY_PASSING (F-405).
in_scope == 0 is E-01 (exit 3), so `conformance` itself is always defined on exit 0/1.
Shape rules (F-016): `by_family` ALWAYS has exactly the six keys R, C, I, K, E, T in that order,
each with in_scope/passing/ratio, even when in_scope is 0. `by_status` ALWAYS has exactly the
seven in-scope statuses in the order shown, each a count (0 allowed); RETIRED is never a key.
`judge_available` is null when judge == none; with mock or llm it is true when at least one

```

judge call succeeded OR no edge was eligible (vacuously available), and false only when every call failed (E-14). The C-07 example's `// comments` are explanatory and never emitted. Family T counts toward in_scope and conformance like every other family (F-001).

C-08 Markdown report layout (SPEC_CONFORMANCE_REPORT.md)

Specification Conformance Report

****Spec:**** `SPEC.md` · ****Judge:**** none|mock|llm (available|unavailable) · ****Strict:**** on|off
 # the parenthetical is omitted (with its leading space) when `judge\_available` is null, i.e. under --ju

1. Verdict

<one line: the §5.1 summary line, verbatim>

2. Metrics

Metric Value	# conformance a/b (0.xxxx); by-status counts; per-family table;
	# judge_strength and unknown_rate rows only when judge enabled

3. Per-ID evidence

ID Status Statement Source citations Test citations (outcome · verdict)

one row per declared ID in C-07 order; RETIRED rows included, with the ID cell (only) struck through:
 # a RECORDED id's ID cell is `T-48 (recorded)`, and a retired recorded id's is `~~T-48~~ (recorded)` - ;
 # label (R-35, F-404); the verdict position shows the em dash, as for any unjudged case;
 # the Statement cell renders the JSON `title` - the heading text or the table cell - never a section bo
 # this table is byte-identical before and after v1.7 (R-33, T-73); the body the judge saw is in `specch
 # citations rendered as `file:line`; verdict rendered as ASSERTS / EXECUTES_ONLY / UNRELATED / UNKNOWN(
 # or as an em dash (-) when the JSON `verdict` is null (Q-001); a file-level case is rendered with the
 # label `(file)` in place of its empty name (Q-011)

## 4. Dangling citations	# table or "None."
## 5. Stale citations	# table or "None."
## 6. Unattributed results	# table or "None."
## 7. Unrun test citations	# table (ID, file, name) or "None."
## 8. Judge details	# present only when judge enabled: one row per judged edge (verdict non-n # Clause (the JSON `clause`, tail-truncated to 80 characters, em dash wh
## 9. Notes	# bullet list or "None."

C-09 LLM provider configuration

SPECHECK_JUDGE_URL	required with --judge llm	e.g. https://host/v1/chat/completions
SPECHECK_JUDGE_MODEL	required with --judge llm	model identifier passed through verbatim
SPECHECK_JUDGE_API_KEY	required with --judge llm	sent as "Authorization: Bearer <key>"
SPECHECK_JUDGE_TIMEOUT	optional, seconds, default 30, integer 1..300	(else exit 2)

The URL and model MAY appear at INFO; the key MUST NOT appear anywhere, including DEBUG output and error messages (redact to "***").

C-10 Judge instruction text (normative; F-004)

The file speccheck/judge_prompt.md is shipped as package data and its content is exactly the text below (trailing newline, \n line endings). It is sent verbatim as the system message (C-06). Changing it is a spec change: bump this document's version and the file together (v1.7 added the any-clause rule for statements that carry a section body — R-33, D-20; v1.9 made the question clause-first and added the clause field — R-34, D-21).

You are a test-strength judge for a specification conformance checker.

You will receive one JSON object with these fields:

```
id          - a specification ID, e.g. "R-07"
statement   - the normative text of that ID: its title on the first line and, for an ID declared
              by a heading, the section beneath it - prose, tables, and code blocks. A statement
              may have several clauses: a pinned interface, numbered rules, table rows.
file        - the path of one test file
start       - the first line number of one test case in that file
end         - the last line number of that test case (inclusive)
source      - the text of lines start..end, one line per source line, each line prefixed by its
              absolute line number and a tab (e.g. "17<TAB>assert x == 2"); cite those numbers
```

Answer exactly one question: does any assertion in this test case check any one clause of the statement? Pick the clause you judge against and quote it verbatim in "clause".

Reply with one JSON object and nothing else - no prose, no markdown fence:

```
{"verdict": <VERDICT>, "clause": <text>, "evidence": [{"file": <file>, "line": <n>}, ...], "rationale": <rationale>}
```

VERDICT is exactly one of:

```
"ASSERTS"      - the test contains at least one assertion (assert statement, assertion method,
                  expected-exception context, or equivalent) whose expected value or condition
                  corresponds to the clause. You MUST list every such assertion line in evidence.
"EXECUTES_ONLY" - the test runs code the clause describes but no assertion checks it
                  (assertions absent, trivial, or about something else).
"UNRELATED"    - the test does not exercise any clause of the statement.
"UNKNOWN"      - you cannot decide from the source given. Prefer UNKNOWN over guessing.
```

Rules:

- clause is a verbatim excerpt of the statement, at least 12 characters and at most 280; copy it, do not paraphrase. For UNRELATED and UNKNOWN, clause is "".
- A statement with several clauses is asserted when any one of them is; it is not required to be asserted whole by one test.
- Every evidence line number MUST be between start and end inclusive, in the given file.
- Cite only lines that exist in source. Do not invent lines.
- rationale is one sentence, at most 280 characters.
- Judge only the given test case. Do not assume what other tests do.
- A test that mentions the ID in a comment or string is not evidence of asserting it.

C-11 Judge progress indicator (R-30)

The indicator is one physical line on stderr, redrawn in place. It is written directly to the stderr stream, **not** through the `speccheck` logger, so the §5.3 logging level is unaffected and the line never carries a level prefix. While it is displayed — from the first draw to the erase — the indicator owns stderr: nothing else is written to that stream (F-201). Every write is one of the two byte sequences below, each issued as a single atomic write (F-203, F-205); no terminal escape sequence is ever used, so the indicator renders on any console that honours a carriage return:

```
draw:  "\r" + <line> + " " * (W - len(<line>))    # W = length of the longest <line> drawn so far in
erase:  "\r" + " " * W + "\r"                    # what remains on stderr after the judge stage is
```

```
<line> ::= "judge: [" <bar> "]" " <done> "/" <total> " edges " <elapsed> " elapsed ~" <left> " left"
<bar>    exactly 20 cells: k times "#" followed by (20 - k) times "-"
<done>    decimal integer, edges whose verdict is determined (received, coerced to UNKNOWN, or budget-s
<total>   decimal integer, number of eligible edges (I-010)
```

<elapsed> M:SS (minutes without upper bound, seconds zero-padded to two digits)
 <left> M:SS, or "?:??" while done == 0

Regex <line> MUST match (T-62), before padding:

```
^judge: \[[#-]{20}\] \d+/\d+ edges \d+:\d{2} elapsed ~(\d+:\d{2}|\?:\?:\?) left$
```

The padding exists because <left> can shrink (~12:00 left → ~9:59 left); padding every draw to the widest line so far means a plain \r never leaves residue, and the erase needs no erase-to-end-of-line escape (F-205).

With n the number of eligible edges, d the number determined so far, and t the wall-clock seconds since t_0 , the filled cell count and the estimate of time remaining are

$$k = \left\lfloor \frac{20d}{n} \right\rfloor$$

$$\text{left} = \frac{t}{d}(n - d) \quad \text{for } d > 0$$

t_0 is the instant of the first draw — the start of the judge stage, immediately before the first request is issued — so the first draw shows 0:00 elapsed. K-12’s budget deadline keeps its own origin, the moment the first request is issued, which is $\geq t_0$ (F-202). left is undefined (rendered ?:??) while $d = 0$; the line is never drawn when $n = 0$ (E-36: no request is sent). Both durations are floored to whole seconds before rendering, with $M = \lfloor s/60 \rfloor$ and $SS = s \bmod 60$. The line is pure ASCII (the cells are # and -, never Unicode block characters) so it renders under any locale (R-29 rationale). It carries no file name, statement text, prompt, response, or key (I-007). The final state ($d = n$, a full bar) is always drawn before the erase.

5. Interface specification

5.1 CLI (speccheck), the only surface

```
speccheck check --spec SPEC.md [--src DIR]... [--tests DIR]... [--results junit.xml]
               [--root DIR] [--out DIR] [--judge none|mock|llm] [--strict]
               [--judge-concurrency N] [--judge-budget SECONDS] [--max-unknown FRACTION]
               [--progress auto|always|never] [--verbose [INFO|DEBUG]]
speccheck --self-check [--verbose [INFO|DEBUG]]
speccheck --version
speccheck --help
```

Subcommand / flag	Behavior	Exit
check	Run the §3.1 pipeline; write both reports; print the summary line.	0 conforming, 1 not, 2 usage, 3 input contract
--spec FILE	Required. Path to the specification; decoded as UTF-8 with errors="replace" (a Note is recorded if any byte was replaced). Missing flag or unreadable file → usage error.	2
--src DIR	Repeatable; default src if it exists, else none. Non-directory → usage error.	2
--tests DIR	Repeatable; default tests if it exists, else none. Non-directory → usage error.	2

Subcommand / flag	Behavior	Exit
<code>--results FILE</code>	Optional JUnit XML. Absent → every cited ID is at most UNVERIFIED. Unreadable → usage error; malformed → E-05.	2 / 3
<code>--root DIR</code>	Base for relative paths in reports; default <code>..</code> . All of <code>--spec/--src/--tests/--results/--out</code> MUST resolve inside it (E-09).	2
<code>--out DIR</code>	Where the two reports go; default <code>..</code> . MUST resolve inside <code>--root</code> (E-09; F-002). Created if missing; not writable → E-18.	2 / 3
<code>--judge MODE</code>	<code>none</code> (default), <code>mock</code> , <code>llm</code> . Any other value → usage error.	2
<code>--strict</code>	Apply R-15 and, with <code>--judge llm</code> , R-28.	—
<code>--max-unknown FRACTION</code>	Threshold for R-28; default 0.2; a float in <code>[0, 1]</code> , else usage error. Accepted with any judge mode; consulted only under <code>--strict --judge llm</code> .	2
<code>--judge-concurrency N</code>	K-06; default 4, integer 1..32, else usage error. Ignored unless <code>--judge llm</code> .	2
<code>--judge-budget SECONDS</code>	K-12; default 0 (unlimited), integer 0..86400, else usage error. Ignored unless <code>--judge llm</code> .	2
<code>--progress MODE</code>	R-30, C-11. <code>auto</code> (default): draw the judge progress indicator iff <code>sys.stderr.isatty()</code> is true and verbosity is not <code>DEBUG</code> ; <code>always</code> : draw it even when <code>stderr</code> is not a TTY (still suppressed under <code>--verbose DEBUG</code> , E-39); <code>never</code> : never draw it. Any other value → usage error. Ignored unless <code>--judge llm</code> .	2
<code>--verbose [LEVEL]</code>	§5.3. Bare = <code>INFO</code> . <code>LEVEL</code> other than <code>INFO/DEBUG</code> → usage error.	2
<code>--self-check</code>	Copy the packaged fixture <code>speccheck/_selfcheck/</code> (a byte-identical copy of the §9.8 golden fixture <code>fixtures/target/</code> , its <code>golden/</code> directory included; F-107) into a fresh temporary directory <code><tmp></code> ; install a socket guard; then run, in the same process , exactly <code>check --spec SPEC.md --src src --tests tests --results junit.xml --judge mock --strict --root <tmp> --out <tmp>/out</code> with the working directory set to <code><tmp></code> (Q-003). The inner check is expected to exit 1 (the fixture has planted defects) and that exit code is not the self-check's result; compare <code><tmp>/out/speccheck.json</code> and <code><tmp>/out/SPEC_CONFORMANCE_REPORT.md</code> to <code><tmp>/golden/</code> byte-for-byte; remove <code><tmp></code> ; print <code>self-check: ok</code> when both match, else one line naming the first mismatch or failure (F-011).	0 / 1
<code>--version</code>	Print <code>speccheck <semver></code> and exit.	0

Summary line (stdout, exactly one line, only on exit 0/1):

`speccheck: <STATUS> - <passing>/<in_scope> passing (<pct>%), <failing> failing, <skipped> skipped, <weal`

The line above is shown unwrapped on purpose: it is one physical line, ASCII only, terminated by a single `\n`, and written to stdout as UTF-8 regardless of locale (R-29; F-010). `<STATUS>` is `CONFORMING` (exit 0) or `NOT CONFORMING` (exit 1); `<pct>` is `conformance × 100` rounded half-even to one decimal; `<judge-suffix>` is empty unless R-28 forced exit 1, in which case it is `(unavailable)` or `(unknown_rate <rate> > max_unknown <max>)` with both numbers as in the JSON; when both R-28 reasons hold the suffix is `(unavailable)` (Q-004). The seven counts after the ratio are the seven in-scope statuses of C-07 by `_status`, each exactly once (Q-005).

Regex the line MUST match (T-44):

```
^speccheck: (CONFORMING|NOT CONFORMING) - \d+/\d+ passing \(\d+\.\d%\), \d+ failing, \d+ skipped, \d+ w
```

5.2 GUI

None. This system has no graphical surface, and O-3 explicitly leaves one out of v0.1.

5.3 Diagnostics and verbosity contract

- **Quiet by default.** With no `--verbose`, the process writes nothing to stderr on exit 0/1 and only the usage/contract error message on exit 2/3. Sole exception: the [C-11](#) progress indicator under `--judge llm` ([R-30](#)), which is written directly to the stream, is redrawn in place, and is erased before the judge stage ends — so what *remains* on stderr at exit is still nothing (or only the error message). It is never routed through the logger.
- **Syntax.** `--verbose` alone $\equiv$ `--verbose INFO`; `--verbose INFO` and `--verbose DEBUG` are accepted; any other value is a usage error (exit 2).
- **INFO** (stderr): one line per pipeline stage with counts and elapsed milliseconds (files scanned, IDs declared, citations found, test cases attributed, results joined, edges judged), the judge mode, and for the LLM judge the URL and model. INFO MUST NOT include file contents, statement text, judge prompts, judge responses, or the API key.
- **DEBUG** (stderr): INFO plus, per judged edge, the full `JudgeRequest` JSON as sent and the raw provider response as received, each prefixed `judge>` / `judge<` on its own line. The API key is redacted. DEBUG MUST NOT change stdout or either report file. Because these lines are emitted *during* the judge stage, the progress indicator is not drawn at DEBUG regardless of `--progress` ([E-39](#)); the per-edge lines are the progress record at that level.
- **Mechanism.** Python logging with a single stderr `StreamHandler`; logger name `speccheck`; format `%(levelname)s %(message)s`. Level is `ERROR` by default and `INFO/DEBUG` per flag, so nothing below `ERROR` reaches stderr without `--verbose` (F-014). The [E-10..E-13](#) and [E-27..E-30](#) Notes are logged at `INFO` (they are always in the report's Notes regardless of level); nothing is ever logged at `WARNING`; the exit-2/3 message is logged at `ERROR`. While the [C-11](#) indicator is displayed, the logger emits nothing: the judge stage's INFO lines (judge mode, URL and model, the stage summary) are written after the erase ([K-13](#); F-201).

5.4 Exit codes

Code	Meaning	Reports written?
0	CONFORMING: no in-scope ID is FAILING; and if <code>--strict</code> , every in-scope ID has status <code>PASSING</code> , <code>dangling</code> and <code>stale</code> are empty, and — with <code>--judge llm</code> only — <code>judge_available</code> is <code>true</code> and <code>unknown_rate</code> $\leq$ <code>max_unknown</code> (R-28). Beyond R-28 , status is the only judge-related input (F-003).	yes
1	NOT CONFORMING: anything else the report can express	yes
2	Usage error (bad flag/value, missing required, path outside root, missing judge env)	no
3	Input contract violation (E-01 , E-02 , E-03 , E-05 , E-18)	no

Any uncaught exception MUST also map to 3 with a one-line message; a traceback is printed only under `--verbose DEBUG`. `KeyboardInterrupt` is treated the same way, with the message `interrupted` ([E-41](#)).

6. Invariants (must hold in every valid implementation)

ID	Invariant
I-001	Read-only inputs. No file under <code>--root</code> other than the two report files under <code>--out</code> — and the <code>--out</code> directory itself, if it did not exist — is created, modified, or deleted by a run, and no temporary of §3.1 survives a run; no file outside <code>--root</code> is written at all. Sole exception: <code>--self-check</code> creates, uses, and removes one fresh temporary directory (F-011); it writes nowhere else.
I-002	Determinism. For <code>--judge none</code> and <code>--judge mock</code> , identical inputs (bytes of every file read, <code>argv</code> , <code>env</code>) produce byte-identical report files and summary line, on any OS. A <code>SPEC.md</code> that differs from another only in <code>\r\n</code> versus <code>\n</code> line endings counts as identical input (C-01 Lines rule; F-301).
I-003	Exactly-once reporting. Every declared ID (retired included) appears exactly once in <code>ids</code> and exactly once in the Markdown per-ID table; no undeclared ID ever appears there.
I-004	Downgrade-only judge. For every ID, <code>status_with_judge</code> $\in$ $\{\text{status_without_judge}\} \cup (\{\text{WEAKLY_PASSING}\} \text{ iff } \text{status_without_judge} == \text{PASSING})$. Disabling the judge never changes any status except <code>WEAKLY_PASSING</code> $\rightarrow$ <code>PASSING</code> .
I-005	Grounded verdicts, on both sides. Every verdict in a report carries an <code>evidence</code> list (possibly empty), every <code>ASSERTS</code> carries ≥ 1 entry, and every evidence entry points to a line inside the judged test case’s span in the judged file; and every <code>ASSERTS</code> or <code>EXECUTES_ONLY</code> carries a <code>clause</code> that is <code>LOCATED</code> in the judged ID’s statement per K-15 (v1.9, R-34).
I-006	Network boundary. With <code>--judge none/mock</code> , no socket is opened for the lifetime of the process.
I-007	Secret and payload hygiene. The API key never appears in <code>stdout</code> , <code>stderr</code> , or either report. <code>INFO</code> never contains file contents, statements, prompts, or responses.
I-008	Total metrics. Every ratio is either a number in <code>[0, 1]</code> or <code>null/n/a</code> ; no run raises on a zero denominator.
I-009	Exit $\equiv$ report. <code>exit_code</code> in <code>speccheck.json</code> equals the process exit status, and both are computable from the rest of the JSON plus <code>strict</code> by the §5.4 rule.
I-010	One judge call per edge. The judge is invoked at most once per (test case, ID) edge per run, and only for edges whose ID is <code>PASSING</code> after C-05 step 4, is not <code>RECORDED</code> (R-35), and whose test outcome is <code>passed</code> .
I-011	Family-safe numbering. ID normalization is injective within a family: R-7, [R-07] (#R-07), R-007 map to one ID; [R-07] (#R-07) and [C-07] (#C-07) never collide.

7. Constraints (precise and measurable)

ID	Constraint
K-01	Usage errors exit 2; input-contract violations exit 3; conformance outcomes exit 0/1; no other exit codes exist.
K-02	Files larger than 2 MiB (2,097,152 bytes) under <code>--src/--tests</code> are skipped with a Note (E-10); binary files (a 0x00 byte within the first 8192 bytes) are skipped silently (E-29); text files that are not valid UTF-8 are decoded with <code>errors="replace"</code> and noted (E-11).
K-03	Directories named <code>.git</code> , <code>.hg</code> , <code>.svn</code> , <code>node_modules</code> , <code>__pycache__</code> , <code>.venv</code> , <code>venv</code> , and any directory whose name starts with <code>.</code> are never descended into; symbolic links to files or directories are never followed (E-30).

ID	Constraint
K-04	ID numbers are 1–3 digits; a token with 4+ digits is not an ID.
K-05	LLM judge timeout is <code>SPECHECK_JUDGE_TIMEOUT</code> seconds (default 30) per edge, wall-clock, measured from the moment the request is issued to the moment the full response body has been received, and covering every phase in between — DNS resolution, TCP connect, TLS handshake, request send, response receive (Q-010).
K-06	Exactly one HTTP request per judged edge; no retries, no batching. Requests MAY be issued concurrently, at most <code>--judge-concurrency</code> at a time (default 4, integer 1..32); output order is fixed by C-07 and MUST NOT depend on completion order (F-110).
K-12	<code>--judge-budget SECONDS</code> (default 0 = unlimited; integer 0..86400) bounds the wall-clock spent in the judge stage, measured from the first request issued. The deadline is <code>start + budget</code> . A request is <i>started</i> when it is issued; no request is issued at or after the deadline; requests in flight at the deadline are allowed to complete (each still bounded by K-05) and their verdicts count. Every edge not started before the deadline receives UNKNOWN with rationale <code>judge: budget</code> and <code>coerced: true</code> , and one Note records how many (F-110, Q-010).
K-13	The progress indicator (C-11) is drawn once when the judge stage starts (with <i>d</i> = 0, 0:00 elapsed); every determined verdict is reflected by a draw within 100 ms of its determination (a burst of verdicts MAY be coalesced into one draw); it is redrawn at least once per second of wall-clock while any request is in flight so that <code>elapsed</code> and <code>left</code> keep moving; and it is never drawn more than 10 times per second. Each draw and the erase is one atomic write, serialized across threads. The bar has exactly 20 cells. From the first draw to the erase nothing else is written to stderr (F-201). The indicator is erased exactly once, when the judge stage ends, after the final state has been drawn and before the <code>report</code> stage begins — before any INFO stage line, Note, error message, or the summary line is written.
K-14	A statement (C-01: title, newline, section body) is at most 16,384 bytes of UTF-8. A longer statement is truncated to the longest prefix of whole lines whose UTF-8 length, joined by <code>\n</code> , is $\leq 16,384$ — the title line is always kept — followed by <code>\n</code> and the marker line ... (<code>statement truncated by speccheck at [K-14] (#K-14)</code>) (U+2026, one space, then ASCII); the marker does not count toward the cap. The truncated text is what <code>SpecId.text</code> , the judge, and the JSON carry; <code>title</code> is unaffected; one Note <code>statement truncated at [K-14] (#K-14): <ID></code> is recorded per truncated ID (E-46). Table declarations never approach the cap. Reference points: the largest contract of the MonteCarloPi spec is 2.7 kB and the largest in this document is C-03 at 8.4 kB, so neither T-48 nor any known spec is truncated; the cap is a bound on the judge payload (about 4 k tokens per edge at most), not a working limit (D-20).
K-15	A <code>clause</code> is LOCATED when, after collapsing every run of whitespace in both strings to one space, trimming the clause (F-406), and cutting it to its first 280 characters, it is a case-sensitive substring of the statement and is at least 12 characters long — or the statement itself is shorter than 12 characters and the clause equals it (an empty statement is matched by an empty clause). The cut, not truncated-with-ellipsis, form is what is recorded (a prefix of a located excerpt is still located).
K-07	<code>rationale</code> ≤ 280 characters after truncation; single line (newlines replaced by spaces).
K-08	Kernel performance, measured on the reference machine named in <code>SPEC_BUILD_REPORT.md</code> (CPU model, RAM, OS, Python build, run in isolation): <code>check --judge mock</code> on the §9.8 golden fixture completes in ≤ 2 s wall-clock, and a generated 10,000-file tree with 100 declared IDs in ≤ 60 s. Both bounds are recorded, not CI-gated (F-017).
K-09	JSON output is UTF-8, <code>indent=2</code> , <code>ensure_ascii=False</code> , sorted per C-07 (not alphabetically), trailing newline; Markdown output is UTF-8 with <code>\n</code> line endings.

ID	Constraint
K-10	<code>--version</code> prints a PEP 440 version equal to the package metadata version.
K-11	<code>--max-unknown</code> defaults to 0.2, accepts a decimal in [0, 1] (parsed as a Decimal from its literal text), and is compared to the quantized <code>unknown_rate</code> of C-07 (Decimal, 4 places, ROUND_HALF_EVEN; Q-009); equality passes; a null <code>unknown_rate</code> (zero judged edges) never exceeds it (F-105).

8. Edge cases and failure semantics

ID	Case	Semantics
E-01	Spec declares zero IDs (or all declared IDs are retired)	Exit 3, message <code>spec declares no in-scope IDs</code> ; no reports written.
E-02	Same ID declared twice (both non-retired, or both retired)	Exit 3, message names the ID and both lines.
E-03	ID declared retired and also declared non-retired (any order)	Exit 3, message names the ID and both lines (a spec must retire <i>or</i> keep an ID, not both).
E-04	ID declared inside a fenced code block in <code>SPEC.md</code>	Not a declaration; ignored silently (spec templates contain example IDs).
E-05	<code>--results</code> file is not well-formed XML, or root is neither <code>testsuites</code> nor <code>testsuite</code> , or a <code><testcase></code> lacks <code>name</code>	Exit 3, message <code>results: <reason></code> .
E-06	Duplicate (<code>classname</code> , <code>name</code>) in results	Single outcome = worst of them (C-04); a Note records the count.
E-07	Result <code><testcase></code> joins no attributed test case	Listed under <code>unattributed_results</code> ; not counted for any ID; exit unaffected.
E-08	Test case cites an ID but has no result (results absent, or case not run)	Case listed under that ID's <code>unrun</code> ; ignored in C-05 step 4; if <i>no</i> citing case has a result → UNVERIFIED.
E-09	Any of <code>--spec/--src/--tests/--results</code> and <code>dir</code> resolves outside <code>--root</code> (after symlink resolution)	Exit 2, message <code>path outside --root: <path></code> . A <code>results</code> and <code>dir</code> path that is itself a symlink and resolves inside <code>--root</code> is used in resolved form and descended; it is not an E-30 skip (Q-007).
E-10	File > 2 MiB under a scan root	Skipped; Note <code>skipped 1 file over 2 MiB: <path></code> .
E-11	A scanned file — or <code>SPEC.md</code> itself — is not valid UTF-8	Decoded with replacement; Note; citations on replaced bytes are impossible (token requires ASCII) so no false citations arise.
E-12	<code>.py</code> test file fails to parse	Whole file is one file-level case; Note <code>parse fallback: <path></code> .
E-13	Test citation lands in a file-level case (module docstring, helper, fixture)	Attributed to the file-level case; that case has no JUnit outcome by construction (name ""), so it can only ever contribute UNVERIFIED; the JSON carries <code>"name": ""</code> and the Markdown renders the label (file) (Q-011).

ID	Case	Semantics
E-14	Judge provider unreachable, times out, or returns HTTP ≥ 400	Verdict UNKNOWN, coerced: true, rationale judge: unavailable / judge: timeout / judge: http <code>; judge_available set false if <i>every</i> call failed; exit unaffected unless --strict (E-32).
E-15	Judge returns non-JSON or JSON lacking verdict	UNKNOWN, coerced: true, rationale judge: malformed response.
E-16	Judge returns ASSERTS with no evidence, or evidence outside the span / in another file	UNKNOWN, coerced: true, rationale judge: ungrounded. When the same reply also fails K-15, E-48 wins (C-06 rule order, F-402). The raw answer is available only at DEBUG.
E-17	Judge returns a valid verdict for an ID whose test span contains no code at all (empty function)	Verdict recorded as returned; the kernel does not second-guess content, only grounding.
E-18	--out cannot be created, or a temporary or final report file cannot be written or renamed	Exit 3; message out: <reason>; on exit neither final report from this run exists and no .tmp from this run remains. Because renames happen JSON-then-Markdown (§3.1), the only stale combination an operator can find is: a previous run's SPEC_CONFORMANCE_REPORT.md with no speccheck.json (F-106).
E-19	Both --src and --tests empty/absent	Run proceeds; every in-scope ID is UNCITED; exit 1 (or 0 only if in_scope == 0, which is E-01).
E-20	Dangling citation of a token that <i>looks</i> retired in code (~~R-99~~ in a comment)	Strikethrough is only meaningful in SPEC.md; in code it is a plain citation of R-99 → dangling.
E-21	--judge llm with a required env var missing	Exit 2, message names the variable, key value never echoed.
E-22	Test file cites the same ID on several lines of one case	One edge; lines lists every line.
E-23	--spec, --results, or a report path lies under a --src/--tests root (e.g. --src ., or --out inside src/)	Those files are not scanned and produce no citations; no Note; a second run over the same inputs is byte-identical to the first (F-002).
E-24	JUnit name carries a parametrization suffix (test_x[3-True]), or several <testcase> join one case	Suffix stripped for the join; each raw result kept in results; the case's outcome is the worst of them (F-005).
E-25	A T id is cited in source files but by no test case	UNCITED (source citations are evidence only for family T); listed under src in the report (F-001).
E-26	An ID has judge verdicts of both ASSERTS and EXECUTES_ONLY on passed edges	Status PASSING; passes --strict; the EXECUTES_ONLY edge remains visible in §8 of the report (F-003).

ID	Case	Semantics
E-27	A result's classname suffix-matches two test cases equally (<code>tests/a/test_core.py</code> and <code>tests/b/test_core.py</code> , result classname <code>test_core</code>)	Unattributed; one Note names the result and all candidates; neither case receives the outcome (F-006).
E-28	A <code>test*</code> function (underscore or not) is a method of a class that is neither <code>Test*</code> -named nor a <code>*TestCase</code> subclass, or of a nested class	Not a test case; its citations are file-level; one Note per file lists the undelimited names (F-007, F-102).
E-29	A file under a scan root contains a <code>0x00</code> byte in its first 8192 bytes	Skipped silently: no citations, no test case, no Note (F-008).
E-30	A symbolic link (file or directory) is encountered <i>during descent</i> under a scan root	Not followed; one Note per run: <code>skipped N symlink(s)</code> (F-008). A symlink that is itself a command-line root is E-09's case, not this one (Q-007).
E-31	A bold ID token appears in a table cell other than the first, or in a heading after other tokens	Not a declaration (F-009). It is also not a citation: <code>SPEC.md</code> is never scanned for citations.
E-32	<code>--strict --judge llm</code> and either <code>judge_available == false</code> or <code>unknown_rate > max_unknown</code>	Exit 1 even if every status is <code>PASSING</code> ; <code>strict_judge_failure</code> set to <code>"unavailable"</code> if the judge was unavailable (whether or not the rate also exceeds), else <code>"unknown_rate"</code> ; summary line carries the matching suffix (F-012, Q-004). Without <code>--strict</code> , or with <code>--judge none mock</code> , no effect.
E-33	A line contains <code>speccheck:ignore</code> , or a file's first three lines contain <code>speccheck:ignore-file</code>	The line yields no citations / the file is not scanned; ignored files are counted in one Note (F-013).
E-34	A <code>.speccheck.json.*.tmp</code> or <code>.SPEC_CONFORMANCE_REPORT.md.*.tmp</code> from a killed run exists under <code>--out</code> , which lies under a scan root	Never scanned (C-03); deleted at §3.1 step 1; the run's reports are byte-identical to a run without the leftover (F-103).
E-35	<code>--judge llm</code> and the <code>--judge-budget</code> deadline passes with edges not yet started	Each such edge is <code>UNKNOWN</code> , <code>coerced: true</code> , rationale <code>judge: budget</code> ; in-flight requests complete and count; one Note <code>judge budget exhausted: N edge(s) unjudged</code> ; <code>unknown_rate</code> counts them; R-28 applies as usual (F-110, Q-010).
E-36	<code>--judge llm</code> with zero eligible edges	No request is sent; <code>judge_available</code> is <code>true</code> ; <code>unknown_rate</code> is <code>null</code> and never trips R-28 (F-105).

ID	Case	Semantics
E-37	A <code>tests[]</code> entry whose case was not judged: judge disabled, or the ID not <code>PASSING</code> after C-05 step 4, or the ID <code>RECORDED</code> (R-35), or the outcome not <code>passed</code> — this row owns the condition; C-05 , C-07 and T-34 cite it (F-403)	JSON <code>verdict</code> is <code>null</code> (key present); Markdown renders an em dash in the verdict position; the entry never appears in report §8 (Q-001).
E-38	Two or more Notes are produced in one run	Emitted in ascending Unicode code-point order of their full text, in both reports (Q-002).
E-39	<code>--judge llm</code> with <code>stderr</code> not a TTY (CI log, redirected file, pipe) under <code>--progress auto</code> ; or <code>--verbose DEBUG</code> under any <code>--progress</code> value; or <code>--progress never</code> ; or <code>--judge none\ mock</code> under any <code>--progress</code> value	No progress bytes (neither draw nor erase) are written to <code>stderr</code> ; the run is otherwise identical. <code>--progress always</code> overrides only the TTY test, never the <code>DEBUG</code> or judge-mode suppression (R-30).
E-40	The judge stage is cut short while the indicator is displayed: a provider raises out of the stage (should not happen — E-14 .. E-16 <code>coerce</code>), an uncaught exception, or an interrupt (E-41)	The erase sequence is written before the exit-3 message (§5.4, E-41); no partially drawn line remains on <code>stderr</code> . The erase MUST happen in a <code>finally</code> -equivalent path so it cannot be skipped (R-30 , K-13).
E-41	<code>SIGINT</code> / <code>KeyboardInterrupt</code> at any stage	Exit 3 with the one-line message <code>interrupted</code> ; the indicator, if displayed, is erased first (E-40); every §3.1 temporary of this run is removed and no report file from this run remains (a report file already renamed is removed, per §3.1’s failure rule); judge requests in flight are abandoned , not awaited — the process MUST exit within 1 s of the interrupt regardless of K-05 — and edges not yet started are never started (v1.5). No other exit code is produced (K-01); a traceback is printed only under <code>--verbose DEBUG</code> (F-204, D-16).
E-42	A <code>.swift</code> test file whose brace depth goes negative on some line or is non-zero at end of file (counted per C-03 , outside comments and string literals)	Whole file is one file-level case; Note <code>parse fallback: <path></code> (R-31 ; the E-12 rule).
E-43	A <code>.swift</code> function named <code>test*</code> that is neither <code>@Test</code> -attributed nor a direct <code>test*</code> method of an <code>XCTestCase</code> class (a method of another type, a member of a nested class, a free function)	Not a test case; its citations are file-level; one Note per file lists the undelimited names as <code><Chain.name></code> (R-31 ; the E-28 rule).

ID	Case	Semantics
E-44	A declaring table row whose first cell begins with a bold ID form and continues with whitespace-separated decoration <code>(**[K-07] (#K-07)** **[port]**)</code>	Declares the ID; the decoration is dropped; a bold ID inside the decoration is not a declaration; a first cell where the form is followed by a non-whitespace character <code>(**[K-07] (#K-07)**x)</code> declares nothing (R-32).
E-45	Two Swift test cases with the same identifier and classname (overloads by parameter label)	A result with that <code>join_name</code> ties between them: unattributed, one Note names the result and both candidates (C-04; the E-27 rule).
E-46	A heading-declared ID whose statement (title plus section body) exceeds K-14; or whose section body is empty — the heading is followed only by blank lines before the next heading of the same or a higher level, or before end of file	Over the cap: truncated per K-14, marker line appended, one Note <code>statement truncated at [K-14] (#K-14): <ID></code> ; <code>title</code> is unaffected and the Markdown table shows it. Empty body: the statement is the title alone, exactly as through v1.6; no Note. Empty title with a body <code>### [C-04] (#C-04)</code> followed by text): the statement is the body alone (F-305). (R-33)
E-47	A heading-declared ID whose section body contains a deeper heading that itself declares an ID <code>#### [E-09] (#E-09) ...</code> under <code>### [C-01] (#C-01) ...</code> , retired or not), or a table with declaring rows	Those inner declarations are parsed exactly as before, each with its own title, statement, and body; the outer ID's statement includes their text verbatim as context (the judge sees the sub-rows). A declaring heading's body ends only at a heading of level $\leq$ its own, so a deeper declaring heading never ends it (R-33, E-31).
E-48	Judge returns <code>ASSERTS</code> or <code>EXECUTES_ONLY</code> with a <code>clause</code> that is missing, empty, paraphrased, or shorter than 12 characters against a longer statement	<code>UNKNOWN, coerced: true, rationale judge: unlocated clause</code> ; counted in <code>unknown_rate</code> , so a model that cannot quote the statement surfaces through R-28 rather than through silent mis-verdicts (R-34, K-15). The raw answer is available only at DEBUG.
E-49	Judge returns <code>UNRELATED</code> or <code>UNKNOWN</code> with a non-empty <code>clause</code> ; or a <code>clause</code> that is present but not a JSON string (<code>null</code> , a number, an object); or a verdict that any C-06 rule coerces to <code>UNKNOWN</code>	Recorded with <code>clause ""</code> ; a returned <code>UNRELATED/UNKNOWN</code> is not coerced and gets no Note; a non-string <code>clause</code> is treated as absent before the K-15 check, so on an <code>ASSERTS/EXECUTES_ONLY</code> it is E-48 (R-34, F-402).
E-50	The <code>*(recorded)*</code> marker on a declaration whose family is not T (<code> </code> <code>**[R-07] (#R-07)**</code> <code>*(recorded)* </code>)	The id is declared normally and is not <code>RECORDED</code> (<code>recorded: false</code>); one Note <code>recorded marker ignored on <ID></code> : not a T id (R-35).

ID	Case	Semantics
E-51	A RECORDED T id whose citing test case failed, was skipped, has no result, or does not exist	Status by C-05 steps 1–4 exactly as for any T id — FAILING , SKIPPED , UNVERIFIED , UNCITED ; the marker exempts an id from the judge (step 5) and from nothing else, so a recorded test still has to be present and green (R-35 , R-15).

9. Acceptance criteria, tests, and evals

Every test cites, in its docstring or a comment, its own T id and the R/C/I/K/E ids it proves, so that `speccheck` can check itself (§9.9; F-001). Test inputs that contain the literal marker strings of [C-01](#) (`speccheck:ignore`, `speccheck:ignore-file`) MUST live in data files under `tests/data/`, never inline in a test module, so self-application cannot ignore its own tests (F-109). Groups are ordered by pipeline stage.

9.1 Extraction (C-01, [C-02](#))

ID	Test
T-01	Table-cell and heading declarations are both extracted with correct family, number, statement, and line; a <code>SPEC.md</code> containing invalid UTF-8 is decoded with replacement, its ASCII declarations are still found, and a Note is recorded. (R-01 , E-11)
T-70	<code>\ **[K-07](#K-07)** **[port]** \ s \ </code> declares K-07 with statement <code>s</code> ; <code>\ **[R-01](#R-01)** **[R-02](#R-02)** \ s \ </code> declares R-01 only; <code>\ **[K-07](#K-07)**x \ s \ </code> and <code>\ **[K-07](#K-07)** , note \ s \ </code> declare nothing; the retired forms tolerate decoration the same way; heading declarations are unchanged. (R-32 , C-01 , E-44)
T-02	<code>R-7, [R-07](#R-07), R-007</code> in a spec are one ID reported as <code>[R-07](#R-07)</code> ; <code>I-5</code> is reported as <code>[I-005](#I-005)</code> . (I-011 , K-04)
T-03	A token with 4 digits (<code>R-1234</code>) and tokens adjacent to alphanumerics (<code>XR-07</code> , <code>R-07a</code>) are not IDs. (K-04 , C-01)
T-04	Strikethrough declarations yield <code>retired=True</code> ; a later plain declaration of the same ID exits 3. (R-02 , E-03)
T-05	IDs inside fenced code blocks in <code>SPEC.md</code> are ignored, for both <code>```</code> and <code>~~~</code> fences, with and without an info string, and for an unclosed fence running to end of file; a backtick fence opened with an info string is closed by a bare backtick line, is not closed by a tilde line, and a candidate closing line carrying an info string does not close it. (E-04 , C-01)
T-06	Duplicate declaration exits 3 and names both lines. (E-02)
T-55	A bold ID in a second cell, a heading with the ID after other tokens, a separator row, and a row with <code>\ </code> and a backtick-quoted pipe in a cell are parsed per C-01 : only first-cell/first-token forms declare; the statement is the second cell with escapes intact. (E-31 , C-01)

ID	Test
T-72	A spec with <code>### [C-01](#C-01) Widget</code>, a fenced block containing <code>\ **R-99** \ x \ </code> and a <code># comment</code> line, a prose paragraph, a <code>#### note</code> sub-section holding a declaring <code>\ **[E-09](#E-09)** \ ... \ </code> row, a <code>---</code>, then <code>### [C-02](#C-02) ...: C-01's text is Widget</code>, a newline, and every line through the <code>---</code> with its indentation and trailing spaces intact (the <code>#### note</code> section included, <code>C-02's</code> heading excluded); <code>C-01's</code> title is <code>Widget</code>; <code>R-99</code> is not declared (E-04) and <code>E-09</code> is (E-47). A body that takes the statement over K-14 yields a <code>text</code> of whole lines $\leq 16,384$ bytes plus the marker line and one Note naming the ID. A heading with an empty body, and a declaring heading on the last line of the file, each yield <code>text == title</code>; <code>### [C-03](#C-03)</code> with no title and a body yields <code>text</code> equal to the body (F-305). The same spec saved with CRLF line endings yields a byte-identical <code>SpecIndex</code> and <code>speccheck.json</code>; a whitespace-only line after the body's last text line is dropped and one inside the body is kept (F-301). A <code># four-space</code> line inside an indented block neither ends a body nor declares; a bare <code>###</code> ends a body; <code>### [C-05](#C-05) Title ###</code> has title <code>Title</code>; <code>###[C-06](#C-06) x</code> declares nothing (F-304). Property, for every ID of <code>fixtures/target/SPEC.md</code> and <code>fixtures/target-swift/SPEC.md</code>: <code>title</code> equals the second cell (table) or the heading remainder after the ID token (heading), whitespace-collapsed, as computed independently by the test; <code>text == title</code> for every table-declared ID and every heading-declared ID with an empty body; <code>text.startswith(title + "\n")</code> otherwise (F-302). (R-33, C-01, C-02, K-14, E-46, E-47) A spec with no in-scope IDs exits 3 and writes no files. (E-01, I-001)
T-07	

9.2 Citation and attribution (C-03)

ID	Test
T-08	Source citations are recorded per (file, line) for tokens in code, comments, and strings alike. (R-03)
T-09	Python test citations inside <code>def test_*</code> and <code>class Test*</code> methods are attributed to the enclosing case with the correct <code>classname</code> , <code>start</code> , <code>end</code> . (R-04)
T-10	A citation in a module docstring or helper is attributed to the file-level case and reported with name (<code>file</code>). (E-13)
T-11	An unparseable <code>.py</code> test file falls back to one file-level case and produces a Note. (E-12)
T-12	A non-Python test file (<code>.go</code> , <code>.ts</code>) gets file-level attribution. (R-04 , O-2)
T-13	Excluded directories (K-03), oversized / non-UTF-8 files (K-02), binary files, and symlinks are skipped or decoded exactly as specified: oversized and non-UTF-8 produce Notes, binary produces none, symlinks produce one Note per run and are never followed (a symlink cycle terminates). (E-10 , E-11 , E-29 , E-30)

ID	Test
T-56	<code>test_*</code> and <code>testFoo</code> methods of a <code>*TestCase</code> subclass, <code>testFoo</code> methods of a <code>Test*</code> class, and <code>async def test_*</code> functions are delimited with correct spans; a module-level <code>testFoo</code> (no underscore) is not a test case; a <code>test*</code> method of an unrecognized class is not, its citations are file-level, and the file gets an <code>undelimited tests</code> Note. (E-28, C-03)
T-57	A line containing <code>speccheck:ignore</code> yields no citations but still counts toward its case's span; a file with <code>speccheck:ignore-file</code> on line 2 yields no citations and no test cases; a marker on line 4 has no file-level effect; markers inside <code>SPEC.md</code> change nothing; the Note counts ignored files. (R-27, E-33)
T-14	Several citations of one ID in one case yield one edge with all lines listed. (E-22)
T-65	Swift Testing attribution: a <code>.swift</code> file with a doc-commented <code>@Test func</code> at file scope, a <code>@Suite struct Outer</code> holding a <code>@Test("named") func</code> , a <code>@Test(arguments: [...])</code> attribute spread over two lines, a nested <code>@Suite struct Inner</code> with a <code>@Test func</code> , a plain helper, and a <code>func testHelper()</code> with no <code>@Test</code> , plus a <code>@testable import</code> : the cases are named by identifier, their classnames are <code><Module></code> , <code><Module>.Outer</code> , <code><Module>.Outer.Inner</code> , each span starts on the first doc-comment line above its attribute and ends on the function's closing brace; a citation on a doc-comment line, on an attribute continuation line and on a body line is attributed to that case; the helper's citation is file-level; the Note lists <code>Outer.testHelper</code> (E-43); <code>@testable</code> is not <code>@Test</code> . <code>MODULE</code> is the first path component under the <code>--tests</code> root, or the root's own name for a file directly under it. (R-31, C-03, E-43)
T-66	XCTest attribution: <code>final class LegacyTests: XCTestCase</code> with <code>func testAddition()</code> and <code>func helper()</code> , a class nested inside it with a <code>test*</code> method, and a <code>class Plain</code> (no <code>XCTestCase</code>) with <code>func testFoo()</code> : <code>testAddition</code> is a case named <code>testAddition</code> with classname <code><Module>.LegacyTests</code> and a span from its doc comment to its closing brace; the nested and the <code>Plain</code> methods are undelimited with one Note naming <code>LegacyTests.Nested.testNested</code> , <code>Plain.testFoo</code> ; helper produces no Note. (R-31, C-03, E-43)
T-67	A <code>.swift</code> file with an unclosed <code>{</code> is one file-level case with Note <code>parse fallback: <path></code> ; a file with a stray <code>}</code> likewise; a file whose only extra braces are inside <code>"{", "...}..."</code> and <code>// }</code> delimits its cases correctly. (E-42, C-03)

9.3 Results mapping (C-04)

ID	Test
T-15	<code><testsuites></code> and bare <code><testsuite></code> roots both parse; <code>failure/error/skipped</code> children map to the correct outcome. (R-05)
T-16	Classname join accepts both <code>tests.test_core</code> and <code>test_core</code> forms for <code>tests/test_core.py</code> , and rejects <code>unit_test_core</code> for <code>test_core</code> . (C-04)

ID	Test
T-58	With <code>tests/a/test_core.py</code> and <code>tests/b/test_core.py</code> , a result with classname <code>tests.a.test_core</code> joins <code>a</code> only; a result with classname <code>test_core</code> is unattributed with a Note naming both candidates. (E-27, C-04)
T-52	<code>test_x[3-True]</code> and <code>test_x[0-False]</code> both join <code>test_x</code> ; <code>results</code> lists both with <code>param</code> ; the case <code>outcome</code> is the worst of them; a name with no <code>[]</code> is unchanged; everything from the first <code>[]</code> is removed: <code>test_y[a][b] → test_y</code> and <code>test_x[list[int]] → test_x</code> with <code>param list[int]</code> ; a result with empty classname joins the unique case with its <code>join_name</code> and is unattributed when two exist. (E-24, C-04, F-105)
T-68	A SwiftPM <code>junit-swift-testing.xml</code> with <code>freeFunction()</code> under <code>ProbeTests</code> , <code>named()</code> , <code>parameterized(x:)</code> , <code>twoArgs(a:b:)</code> and a <code><skipped></code> <code>disabledOne()</code> under <code>ProbeTests.Outer</code> , and <code>nested()</code> under <code>ProbeTests.Outer.Inner</code> joins each to its C-03 case with <code>param null</code> and the outcomes <code>passed / skipped</code> ; an <code>XCTest <testcase classname="ProbeTests.LegacyTests" name="testAddition"></code> joins the <code>XCTest</code> case; results for overloads <code>f(a:)</code> and <code>f(b:)</code> are unattributed with one Note naming both candidates (E-45); <code>test_x[f(1)]</code> still strips from the first <code>[]</code> and keeps <code>param f(1)</code> . (R-31, C-04, E-45)
T-17	Duplicate (<code>classname</code> , <code>name</code>) collapses to the worst outcome in the order <code>error > failed > skipped > passed</code> . (E-06)
T-18	Results for unknown cases are listed as unattributed and change no status. (E-07)
T-19	Malformed XML and a <code><testcase></code> without <code>name</code> each exit 3. (E-05)

9.4 Status algorithm and metrics (C-05, C-07)

ID	Test
T-20	One fixture per deterministic status — <code>UNCITED</code> , <code>UNTESTED</code> , <code>UNVERIFIED</code> , <code>FAILING</code> , <code>SKIPPED</code> , <code>PASSING</code> , <code>RETIRED</code> — each asserting exactly that status; the <code>UNCITED</code> fixture has empty <code>--src</code> and <code>--tests</code> and exits 1. (R-06, E-19)
T-21	An ID with one failed and three passed citing tests is <code>FAILING</code> ; with all citing tests skipped is <code>SKIPPED</code> ; with some skipped and some passed is <code>PASSING</code> . (C-05 step 4)
T-22	Citing cases absent from results are listed under <code>unrun</code> and do not affect the status. (E-08)
T-23	Dangling and stale citations are listed with file and line; strikethrough in code does not retire anything. (R-07, R-08, E-20)
T-24	<code>conformance</code> , per-family ratios, and counts match hand-computed values on the golden fixture; a family with zero in-scope IDs reports <code>null/n/a</code> , not <code>0.0</code> , and nothing raises. (R-09, I-008)
T-25	Retired IDs are excluded from every denominator and still appear in <code>ids</code> exactly once. (R-02, I-003)
T-53	A T id cited only in a source file is <code>UNCITED</code> ; a T id cited by a passing test is <code>PASSING</code> ; T ids are counted in <code>in_scope</code> , <code>conformance</code> , and <code>by_family.T</code> . (R-25, E-25)

ID	Test
T-77	**[T-01] (#T-01)** *(recorded)* s declares T-01 with <code>recorded == True, title == "s";</code> ### [T-02] (#T-02) *(recorded)* Title declares T-02 recorded with title Title; **[T-03] (#T-03)** *(Recorded)* , **[T-04] (#T-04)** (recorded) and **[T-05] (#T-05)** **[port]** *(recorded)* declare their ids not recorded; **[R-01] (#R-01)** *(recorded)* declares R-01 not recorded with the E-50 Note. With <code>--judge mock</code> and a call-counting stub under <code>--judge llm</code>: a recorded T id cited by a passing test with an assertion-free body is <code>PASSING</code> and its edge is never sent to the judge (<code>verdict null</code>, not counted in <code>unknown_rate</code>'s denominator or the C-11 total), while the same test cited by a non-recorded T id yields <code>WEAKLY_PASSING</code>; a recorded T id with a failing test is <code>FAILING</code>, with a skipped one <code>SKIPPED</code>, with no citing test <code>UNCITED</code> (E-51); with two recorded <code>PASSING</code> ids, one judged <code>PASSING</code> id and one <code>WEAKLY_PASSING</code> id, <code>judge_strength</code> is 1/2 (recorded ids outside the population) while <code>conformance</code> counts all three passing (F-405); the JSON carries <code>"recorded": true</code> after "family" and the Markdown ID cell reads <code>[T-01] (#T-01) (recorded)</code> and a retired recorded id's <code>~~T-03~~ (recorded)</code>; <code>schema_version</code> is the C-07 value. (R-35, C-01, C-02, C-05, C-07, C-08, I-010, E-50, E-51)

9.5 Judge contract (C-06)

ID	Test
T-26	Mock judge returns <code>ASSERTS</code> with one evidence line per assertion token, and <code>EXECUTES_ONLY</code> with no evidence when none is present, including for an empty test body. (R-22 , E-17)
T-69	The mock judge returns <code>ASSERTS</code> with the line as evidence for a Swift span containing <code>#expect(, #require(, XCTAssertEqual(, XCTFail(</code> or <code>Issue.record(</code> (one sub-test each), and <code>EXECUTES_ONLY</code> for a Swift body that only calls code <code>(_ = add(1, 2))</code> . (R-22 , C-06)
T-27	A <code>PASSING</code> ID whose only passed edges are <code>EXECUTES_ONLY</code> becomes <code>WEAKLY_PASSING</code> ; with any <code>ASSERTS</code> edge it stays <code>PASSING</code> and passes <code>--strict</code> ; with only <code>UNKNOWN</code> it stays <code>PASSING</code> . (C-05 step 5, R-11 , E-26)
T-28	Disabling the judge changes no status except <code>WEAKLY_PASSING</code> → <code>PASSING</code> (property test over random fixtures). (I-004)
T-29	A stub provider returning <code>ASSERTS</code> without evidence, evidence outside the span, or evidence in another file is coerced to <code>UNKNOWN</code> with <code>coerced: true</code> and rationale <code>judge: ungrounded</code> . (E-16 , I-005)
T-30	Non-JSON, missing <code>verdict</code> , an unknown verdict string, a raised exception, and a timeout each yield <code>UNKNOWN</code> with the specified rationale. (E-14 , E-15)
T-31	The judge is called exactly once per eligible edge and never for <code>FAILING/SKIPPED/UNVERIFIED/UNTESTED/UNCITED</code> IDs or for skipped/failed test outcomes (call-counting stub). (I-010)
T-32	Rationale over 280 chars is truncated to 277 + ...; newlines become spaces. (K-07)

ID	Test
T-33	LLM provider sends exactly one POST per edge whose body is byte-for-byte the C-06 shape (<code>model</code> , <code>temperature: 0</code> , <code>max_tokens: 4000</code> , system message equal to the C-10 text, user message equal to the request JSON with line-numbered <code>source</code>), with the bearer header, honors the timeout, never retries, and issues at most <code>--judge-concurrency</code> requests at once while output order stays fixed (recorded HTTP stub with a concurrency counter); the URL, model, key and timeout are those of the C-09 variables. (K-05 , K-06 , R-26 , C-09)
T-54	The provider reads <code>choices[0].message.content</code> , accepts a bare JSON object and one wrapped in a <code>```/```json</code> fence, and treats a missing path, a non-200 status, or any other text as E-14/E-15 ; the report carries <code>judge_prompt_sha256</code> equal to the SHA-256 of <code>speccheck/judge_prompt.md</code> , and that file equals the C-10 text. (C-06 , C-10 , R-26)
T-74	Extends T-33 : for a heading-declared ID with a body, the LLM request’s user message carries <code>statement</code> equal to <code>SpecId.text</code> — title, newline, body with its fenced block and indentation, byte for byte — under the unchanged keys <code>{id, statement, file, start, end, source}</code> and the unchanged system message; the C-10 file contains the any-clause rule added in v1.7 and <code>judge_prompt_sha256</code> is its SHA-256 (recorded HTTP stub). (R-33 , C-06 , C-10 , R-26)
T-75	Clause grounding: a stub returning <code>ASSERTS</code> with a clause that is a verbatim excerpt of the statement differing only in line breaks and indentation is accepted and recorded with the collapsed excerpt; a 300-character excerpt is accepted and recorded cut to 280; a paraphrase, an 8-character fragment of a longer statement, a missing key, and <code>""</code> each yield <code>UNKNOWN</code> , <code>coerced: true</code> , <code>judge: unlocated clause</code> , for <code>ASSERTS</code> and for <code>EXECUTES_ONLY</code> alike; an <code>UNRELATED</code> with a clause is recorded with <code>""</code> and not coerced; <code>"clause": null</code> on an <code>UNRELATED</code> is <code>""</code> and not coerced, on an <code>ASSERTS</code> it is E-48 ; an <code>ASSERTS</code> whose clause is unlocated <i>and</i> whose evidence lies outside the span records <code>judge: unlocated clause</code> (rule order); a clause with a leading space that otherwise matches the statement’s first words is <code>LOCATED</code> (K-15 trims); a statement of 9 characters is matched by itself and by nothing shorter; the mock judge’s clause is the collapsed statement’s first 280 characters on every edge of the golden fixture and equals <code>""</code> for an ID whose statement is <code>""</code> ; the LLM request’s system message equals the current C-10 text; the JSON verdict object’s keys are <code>verdict</code> , <code>clause</code> , <code>evidence</code> , <code>rationale</code> , <code>coerced</code> in that order and <code>schema_version</code> is the C-07 value; report §8 shows the clause column with an em dash for <code>""</code> . (R-34 , K-15 , E-48 , E-49 , I-005 , C-06 , C-07 , C-08 , C-10 , R-22)

9.6 Reports and determinism ([C-07](#), [C-08](#))

ID	Test
T-34	<code>speccheck.json</code> validates against the C-07 shape: key order, sort orders, rounding, no absolute paths, no timestamps; <code>by_family</code> has all six keys and <code>by_status</code> all seven in-scope statuses on every fixture including single-family ones; <code>judge_available</code> is <code>null</code> for <code>none</code> , <code>true</code> for <code>mock</code> , <code>true</code> for <code>llm</code> with zero eligible edges, <code>false</code> for <code>llm</code> when every call fails; every <code>tests[]</code> entry carries the <code>verdict</code> key, <code>null</code> on a <code>FAILING</code> ID, on a skipped-outcome edge of a <code>PASSING</code> ID, on every edge of a <code>RECORDED</code> ID (E-37 , R-35), and everywhere under <code>--judge none</code> ; <code>tests[].name</code> is <code>"</code> for a file-level case; a fixture producing five Notes emits them in code-point order in both reports; every ratio has exactly four decimal places and a hand-picked tie value (e.g. $1/8 = 0.125$ at 3 places, 0.00005 at 4) rounds half-even to the quantized Decimal, not to <code>round()</code> 's binary result. (R-13 , R-20 , K-09 , C-07 , E-37 , E-38)
T-35	<code>SPEC_CONFORMANCE_REPORT.md</code> has the nine C-08 sections in order, the header's judge parenthetical reading <code>(available)/(unavailable)</code> for <code>mock/llm</code> and absent for <code>none</code> , one per-ID row per declared ID, retired rows with exactly the ID cell struck through, an em dash in the verdict position for every unjudged case and <code>(file)</code> for every file-level case, and every <code>file:line</code> in it also present in the JSON. (R-12 , I-003 , E-37 , Q-011)
T-36	Two runs on the golden fixture with <code>--judge mock</code> produce byte-identical reports and summary lines; the same holds after copying the fixture to a different absolute path, and with <code>--src .</code> and <code>--out</code> inside the source root (the previous run's reports and the spec itself produce no citations), and with a planted <code>.speccheck.json.deadbeef.tmp</code> and <code>.SPEC_CONFORMANCE_REPORT.md.deadbeef.tmp</code> under <code>--out</code> before the run (never scanned, deleted by the run). (R-16 , I-002 , E-23 , E-34)
T-37	Every metric in the report is recomputable from the report's own evidence table plus the results file (the test recomputes them independently). (R-24)
T-38	Only the two report files are created; input trees are byte-identical before and after (hash comparison). (R-19 , I-001)

9.7 CLI, exit codes, diagnostics, boundary (§5)

ID	Test
T-39	<code>exit_code</code> in JSON equals the process exit status for fixtures exercising 0, 1 (non-strict), 1 (strict-only reasons: <code>UNVERIFIED</code> , <code>UNTESTED</code> , <code>UNCITED</code> , <code>SKIPPED</code> , <code>WEAKLY_PASSING</code> , <code>dangling</code> , <code>stale</code>), and 0 under <code>--strict</code> for a mixed <code>ASSERTS/EXECUTES_ONLY</code> ID. (R-14 , R-15 , I-009 , E-26)
T-40	Missing <code>--spec</code> , bad <code>--judge</code> , bad <code>--verbose</code> value, any input or <code>--out</code> path outside <code>--root</code> , and missing <code>judge</code> env each exit 2 with the specified message; the API key value never appears in the message. (K-01 , E-09 , E-21 , R-23 , C-09)
T-41	Bare <code>--verbose</code> $\equiv$ <code>--verbose INFO</code> ; <code>INFO</code> emits stage lines with counts and no statement text, file contents, or prompts; <code>DEBUG</code> emits <code>judge>/judge<</code> lines with the key redacted; <code>stdout</code> is the single summary line in all cases. (R-17 , I-007)

ID	Test
T-42	With no <code>--verbose</code> , stderr is empty on exit 0/1 even when the run produced Notes; with <code>--verbose</code> , each Note appears once at INFO. (§5.3)
T-43	With <code>--judge none</code> and <code>--judge mock</code> , a socket-creation guard installed for the process lifetime is never triggered; <code>speccheck --self-check</code> passes from an installed wheel in an empty working directory, invokes the inner check in-process with exactly the §5.1 argument list (asserted via a recorded <code>Config</code>), prints <code>self-check: ok</code> although its inner check exits 1, leaves no files behind, and writes nothing outside its temporary directory. (R-18, I-006, I-001, Q-003)
T-60	<code>speccheck/_selfcheck/</code> is byte-identical to <code>fixtures/target/</code> (recursive file-by-file comparison, <code>golden/</code> included); the test fails if either side changes without the other. (F-107, §10)
T-44	Summary line matches the §5.1 regex exactly, is pure ASCII, ends in a single <code>\n</code> , is emitted correctly under a C/POSIX locale and a <code>cp1252</code> stdout, carries all seven in-scope status counts including <code>skipped</code> , and its numbers equal the JSON metrics. (R-21, R-29, Q-005)
T-59	Under <code>--strict --judge llm</code> with a provider stub that fails every call — so that <code>judge_available</code> is false <i>and</i> <code>unknown_rate</code> is <code>1.0000 > max_unknown</code> — exit is 1, <code>strict_judge_failure</code> is "unavailable" (not "unknown_rate"), and the summary line ends <code>judge=llm (unavailable)</code> (Q-004); with a stub yielding 3 UNKNOWN of 10 edges and <code>--max-unknown 0.2</code> , exit is 1 with "unknown_rate" and the numeric suffix; with <code>--max-unknown 0.3</code> exit is 0; without <code>--strict</code> neither affects exit; with a fixture that has zero eligible edges, no request is sent, <code>unknown_rate</code> is null, and <code>--strict</code> exits 0. (R-28, K-11, E-32, E-36)
T-61	With <code>--judge-budget 1</code> , <code>--judge-concurrency 2</code> , and a provider stub that sleeps 2 s per call over six edges, exactly the two requests issued before the deadline complete and are judged (in-flight requests are not cancelled); the remaining four are UNKNOWN with rationale <code>judge: budget</code> and <code>coerced: true</code> ; the Note reports 4; <code>unknown_rate</code> includes them; with <code>--judge-budget 0</code> all six are judged. (K-12, E-35, Q-010)

ID	Test
T-62	With <code>--judge llm</code>, <code>--progress always</code>, a provider stub over six edges that sleeps 0.3 s per call, <code>--judge-concurrency 2</code>, and stderr captured: splitting the capture on <code>\r</code> yields only C-11 draw and erase sequences (after stripping each segment's trailing space padding, every draw matches the C-11 regex); the first draw reads <code>0/6 edges, 0:00 elapsed, ?:??</code>; <code><done></code> is non-decreasing across draws; a draw with <code>6/6</code> and a full <code>20-#</code> bar exists and is the last draw; the capture ends with the erase sequence (<code>\r</code>, <code>W</code> spaces, <code>\r</code>) and no <code>\x1b</code> byte appears anywhere; every draw is padded to the width of the widest draw so far; the number of <code>#</code> in each draw equals $\lfloor 20d/6 \rfloor$ for its <code><done></code>; with a stub that sleeps 2.5 s on a single edge, at least two draws show <code>0/1</code> with distinct <code>elapsed</code> values (the 1 s tick); with 32 edges answered instantly at <code>--judge-concurrency 32</code>, no more than 10 draws occur in any one second and the final draw still shows <code>32/32</code> (coalescing); with <code>--verbose INFO</code> added, no <code>INFO</code> line occurs between the first draw and the erase and the judge stage's <code>INFO</code> lines follow the erase; stdout is the single summary line and both report files are byte-identical to a <code>--progress never</code> run over the same stub. (R-30, C-11, K-13)
T-63	No progress bytes reach stderr when: stderr is not a TTY under <code>--progress auto</code> (the default in a subprocess capture); <code>--progress never</code>; <code>--verbose DEBUG</code> with <code>--progress always</code>; <code>--judge mock</code> or <code>--judge none</code> with <code>--progress always</code>. With <code>--progress auto</code> and a stderr whose <code>isatty()</code> returns <code>True</code>, the indicator is drawn. <code>--progress</code> with any other value exits 2. Under <code>--progress always</code> a stub that raises <code>KeyboardInterrupt</code> mid-stage leaves the erase sequence as the last stderr bytes before the <code>interrupted</code> message (E-41). (R-30, E-39, E-40, K-01)
T-64	A <code>KeyboardInterrupt</code> raised by a provider stub mid-judge, and one injected between §3.1 steps 3 and 4 (after the JSON rename, before the Markdown rename), each exit 3 with stderr exactly <code>ERROR interrupted</code> (plus a traceback only under <code>--verbose DEBUG</code>), write nothing to stdout, leave no <code>.tmp</code> and no <code>speccheck.json</code> from this run under <code>--out</code>, and leave a previous run's reports intact; the JSON <code>exit_code</code> is never consulted (no report exists). With six edges, <code>--judge-concurrency 2</code>, and a stub that sleeps 5 s per call, an interrupt delivered to the main thread 0.5 s in ends the run in under 2 s with at most three requests ever started (in-flight requests abandoned, queued edges never issued). (E-41, K-01, I-001, K-05, §3.1)
T-45	Unwritable <code>--out</code> exits 3 with nothing written; a failure injected after the JSON rename and before the Markdown rename exits 3 with no <code>.tmp</code> left, no <code>speccheck.json</code>, and the previous run's <code>SPEC_CONFORMANCE_REPORT.md</code> intact (the one stale combination E-18 permits); leftover temporaries from an earlier run are deleted first; on success no <code>.tmp</code> remains and the two temporaries used carried the same 8-hex-digit nonce. (E-18, I-001, §3.1)
T-50	<code>speccheck --version</code> prints <code>speccheck <version></code> where <code><version></code> is PEP 440 and equals <code>importlib.metadata.version("speccheck")</code>. (K-10)

9.8 Golden fixture (end to end)

ID	Test
T-46	<code>fixtures/target/</code> — a small self-contained project with its own <code>SPEC.md</code> (≥ 12 IDs across all six families, 2 retired, one heading-declared contract with a multi-clause body $\geq 2,048$ bytes per T-76), <code>src/</code>, <code>tests/</code>, and a checked-in <code>junit.xml</code> — contains planted defects: one UNCITED R, one UNTESTED C, one UNVERIFIED E, one FAILING T, one SKIPPED K, one EXECUTES_ONLY-only test, one dangling citation, one stale citation, one unattributed result, one file-level citation. The golden reports live in <code>fixtures/target/golden/</code> (outside every scan root). <code>speccheck check --spec SPEC.md --src src --tests tests --results junit.xml --judge mock --strict --root fixtures/target --out <fresh tmp></code> produces files byte-identical to <code>golden/speccheck.json</code> and <code>golden/SPEC_CONFORMANCE_REPORT.md</code> and exits 1 (Q-003). (all of §2)
T-47	Removing each planted defect in turn flips exactly the expected row and metric (one sub-test per defect). (R-06, R-24)
T-71	<code>fixtures/target-swift/</code> — a SwiftPM-shaped project with its own <code>SPEC.md</code> (≥ 10 IDs across all six families, one carrying <code>**[port]**</code> decoration), <code>Sources/</code>, <code>Tests/<Module>/</code> holding one Swift Testing file (nested suite, parameterized test, disabled test, doc-comment citations) and one XCTest file, and a checked-in <code>junit.xml</code> that concatenates the <code>junit-swift-testing.xml</code> SwiftPM wrote and an XCTest <code><testsuite></code> in SwiftPM's shape — contains planted defects: one UNCITED R, one UNTESTED C, one UNVERIFIED file-level citation, one FAILING T, one SKIPPED K (the disabled test), one EXECUTES_ONLY-only test, one undelimited <code>test*</code> helper (E-43), one unattributed result. With <code>--judge mock</code> both reports are byte-identical to <code>fixtures/target-swift/golden/</code>, and the summary line is exactly the one recorded in that golden. (R-31, R-32, R-16, R-24)
T-73	Both golden fixtures regenerated for v1.7 and, as properties of the current output: each <code>golden/speccheck.json</code> carries the C-07 <code>schema_version</code> and a title per ID; in <code>fixtures/target/golden/speccheck.json</code> C-01's statement is its title, a newline, and The error message MUST name the dividend. while C-02's (empty body) equals its title; in each <code>golden/SPEC_CONFORMANCE_REPORT.md</code> the Statement cell of every §3 row equals that ID's JSON title, contains no newline, and no row contains the K-14 marker; the set of IDs in the Markdown equals the set in the JSON; <code>speccheck/_selfcheck/</code> is updated in step (T-60) and <code>--self-check</code> prints <code>self-check: ok</code>. The one-time v1.6 $\rightarrow$ v1.7 golden diff (only <code>schema_version</code> and the <code>title</code> keys added, heading-declared <code>statements</code> changed, Markdown unchanged) is recorded in <code>SPEC_BUILD_REPORT.md</code>, not asserted by a test (F-302). (R-33, C-07, C-08, T-46, T-71)

ID	Test
T-76	<code>fixtures/target/SPEC.md</code> declares one heading-declared contract whose SECTION BODY is at least 2,048 bytes and consists of a fenced code block pinning a struct with field comments followed by at least five numbered rules; <code>tests/</code> holds four tests that each assert exactly one of those rules or the struct's shape and cite the contract, one test that calls the code and asserts nothing about it while citing the contract, and one test that asserts a different ID's behaviour while citing the contract; the six are attributed to their own cases and appear in <code>golden/judge_labels.json</code> as <code>ASSERTS ×4</code> , <code>EXECUTES_ONLY ×1</code> , <code>UNRELATED ×1</code> ; the label file has at least 20 entries of which at least 6 are edges of an ID whose statement exceeds 2,048 bytes; under <code>--judge mock</code> the contract is <code>PASSING</code> and the T-46 golden is byte-identical. (T-46 , T-49 , R-34)

9.9 Self-application (recorded, not gating)

ID	Test
T-48 (<i>recorded</i>)	<code>speccheck check --spec SPEC.md --src src --tests tests --results <this suite's junit.xml> --judge mock</code> on this repository reports every R/C/I/K/E/T ID in this document as <code>PASSING</code> . Recorded in <code>SPEC_BUILD_REPORT.md</code> ; it is evidence <i>for</i> the build, not the proof of it — the proof is §9.1–§9.8. (R-24)

9.10 Performance (recorded)

ID	Test
T-51 (<i>recorded</i>)	A benchmark script (<code>tools/bench.py</code> , not collected by <code>pytest</code>) runs <code>check --judge mock</code> on the golden fixture and on the generated 10,000-file tree five times each and prints the median wall-clock; the medians, the reference-machine description, and the pass/fail against K-08 are recorded in <code>SPEC_BUILD_REPORT.md</code> . Not run in CI. (K-08)

9.11 LLM judge evaluation (opt-in, `--judge llm`, not run in CI)

ID	Test
T-49 (<i>recorded</i>)	On the golden fixture’s judged edges (each hand-labeled ASSERTS/EXECUTES_ONLY/UNRELATED), the LLM judge’s non-UNKNOWN verdicts agree with labels at ≥ 0.90 accuracy and <code>unknown_rate</code> ≤ 0.10 in each of three independent runs (no pooling; one failing run fails T-49); all three per-run figures are recorded with model name, date, and <code>judge_prompt_sha256</code> . The labels live in <code>fixtures/target/golden/judge_labels.json</code> , a JSON object mapping " <code><ID> <file>::<name></code> " (the judged edge) to its label, with at least 20 entries of which at least 6 are edges of a statement over 2,048 bytes (T-76), so that a judge which grades a body by its gist fails the run while one that locates the clause passes; the runner is <code>tools/eval_judge.py</code> (not collected by pytest), which runs the check three times and scores each run against the labels (F-210). A run counts toward T-49 only if its recorded <code>judge_prompt_sha256</code> equals the SHA-256 of the current C-10 text; a change to C-10 , or to C-01 ’s statement rule, therefore voids the recorded runs and requires three fresh ones, and the labels are re-read against the current statements before those runs (F-303). (R-10 , R-26)

10. Dependencies and environment

- **Runtime:** Python 3.12; uv for environment and lockfile. Install: `uv sync --extra dev`; with the model judge: `uv sync --extra dev --extra llm`.
- **Kernel dependencies:** none beyond the standard library (`re`, `ast`, `xml.etree.ElementTree`, `json`, `argparse`, `pathlib`, `logging`, `hashlib`).
- **Package data:** `speccheck/judge_prompt.md` ([C-10](#)) and `speccheck/_selfcheck/` (a byte-identical copy of `fixtures/target/` including its `golden/` directory, kept in sync by [T-60](#); F-011, F-107), both included in the wheel.
- **Golden fixture layout:** `fixtures/target/{SPEC.md, src/, tests/, junit.xml, golden/speccheck.json, golden/SPEC_CONFORMANCE_REPORT.md}`; `golden/` is never a scan root and is never written to by a run (Q-003).
- **[llm] extra:** `httpx` (HTTP client with per-request timeout). No provider SDK; the wire format is [C-06](#).
- **[dev] extra:** `pytest`, `pytest-cov`, `hypothesis` ([T-28](#)), `ruff`.
- **Environment variables:** [C-09](#) (only with `--judge llm`). No configuration files are read.
- **Host prerequisites:** none. No network access is required except with `--judge llm`.
- **Commands:**

```
uv sync --extra dev
uv run python -m pytest tests -q --junitxml=junit.xml      # the $9 suite; junit.xml feeds T-48
uv run ruff check src tests
uv run speccheck --self-check
uv run speccheck check --spec SPEC.md --src src --tests tests --results junit.xml --judge mock
```

- **Optional items:**
 - **O-1** LLM judge provider (`--judge llm`, `[llm] extra`). Specified fully in [C-06/C-09](#); gated by flag and extra.
 - **O-2** Additional language adapters for test-case attribution (Go `func Test*`, JS/TS `test(/it()`). Not in v0.1; the fallback path is what v0.1 ships and tests ([T-12](#)). Swift joined the shipped adapters in v1.6 ([R-31](#), [C-03](#)); Go and JS/TS remain optional.
 - **O-3** Any non-CLI surface (GUI, HTTP, editor plugin). Not in v0.1 and not designed for.

11. Traceability matrix (id → where realized)

Until the build exists, “where realized” names the component the §1/§4 design assigns; `spec-build` replaces it with the real module and the real test.

Spec id	Where realized (component / module)	Verified by
R-01	<code>extract.py</code> (declaration scan, UTF-8 replace)	T-01, T-05, T-46, T-72
R-02	<code>extract.py</code> (retired flag), <code>graph.py</code> (denominators)	T-04, T-25, T-46
R-03	<code>extract.py</code> (source scan, C-03 exclusions, binary/symlink filters)	T-08, T-13, T-36, T-46
R-04	<code>attribute.py</code> (Python adapter, Swift adapter, fallback, C-03 exclusions)	T-09, T-10, T-11, T-12, T-36, T-56, T-46, T-65, T-66, T-67
R-05	<code>results.py</code>	T-15, T-16, T-52, T-58, T-46, T-68
R-06	<code>graph.py</code> (C-05 steps 1–4)	T-20, T-21, T-47, T-46
R-07	<code>graph.py</code> (dangling)	T-23, T-46
R-08	<code>graph.py</code> (stale)	T-23, T-46
R-09	<code>graph.py</code> (metrics)	T-24, T-46
R-10	<code>judge.py</code> , <code>judge_mock.py</code> , <code>judge_llm.py</code> (passed-outcome edges only)	T-26, T-31, T-33, T-49, T-46
R-11	<code>graph.py</code> (C-05 step 5)	T-27, T-28, T-46
R-12	<code>report.py</code> (Markdown)	T-35, T-46
R-13	<code>report.py</code> (JSON)	T-34, T-46
R-14	<code>cli.py</code> (exit mapping)	T-39, T-46
R-15	<code>cli.py</code> (<code>--strict</code> , status-only)	T-39, T-27, T-46
R-16	all kernel modules (sorted, no volatile fields)	T-36
R-17	<code>cli.py</code> (logging setup, level <code>ERROR</code> default)	T-41, T-42
R-18	<code>cli.py</code> (<code>--self-check</code> , pinned in-process invocation), <code>speccheck/_selfcheck/</code> , <code>judge_llm.py</code> import isolation	T-43, T-60
R-19	<code>report.py</code> (only writer, temp-and-rename), <code>cli.py</code>	T-38, T-43, T-45
R-20	<code>extract.py</code> , <code>report.py</code> (path normalization)	T-34, T-36
R-21	<code>cli.py</code> (summary line)	T-44, T-59
R-22	<code>judge_mock.py</code> (Python and Swift assertion tokens; prefix clause)	T-26, T-69, T-75
R-23	<code>judge_llm.py</code> , <code>cli.py</code> (env validation, redaction)	T-40, T-41
R-24	<code>report.py</code> (evidence table completeness)	T-37, T-48
R-25	<code>graph.py</code> (C-05 step 2b)	T-53
R-26	<code>judge_llm.py</code> (body, response path, prompt hash), <code>speccheck/judge_prompt.md</code>	T-33, T-54, T-74
R-27	<code>extract.py</code> (ignore markers)	T-57
R-28	<code>cli.py</code> (<code>--strict</code> judge gate), <code>report.py</code> (<code>strict_judge_failure</code>)	T-59
R-29	<code>cli.py</code> (summary line encoding)	T-44
R-30	<code>cli.py</code> (<code>--progress</code> resolution, TTY test), <code>judge.py</code> (progress callback in <code>run_judge</code>)	T-62, T-63
R-31	<code>attribute.py</code> (Swift adapter: type/func lines, attribute block, brace spans, MODULE), <code>results.py</code> (signature strip), <code>judge_mock.py</code> (Swift tokens)	T-65, T-66, T-67, T-68, T-69, T-71
R-32	<code>extract.py</code> (first-cell prefix rule)	T-70, T-71

Spec id	Where realized (component / module)	Verified by
R-33	<code>extract.py</code> (heading title, SECTION BODY, K-14 cap and Note), <code>report.py</code> (title key; Markdown renders title), <code>judge_llm.py</code> (statement passthrough)	T-72, T-73, T-74
R-34	<code>judge.py</code> (clause validation, K-15 matcher, E-48/E-49 coercion), <code>judge_llm.py</code> (clause in the reply), <code>judge_mock.py</code> (prefix clause), <code>report.py</code> (clause key, §8 column), <code>speccheck/judge_prompt.md</code>	T-75, T-76, T-49
R-35	<code>extract.py</code> (RECORDED marker, <code>SpecId.recorded</code>), <code>graph.py</code> (step 5 skip, eligible edges), <code>report.py</code> (recorded key, ID cell)	T-77
C-01	<code>extract.py</code> (ID_RE, fence tracker, row/heading parsers incl. first-cell decoration, RECORDED marker and section bodies, ignore markers)	T-01, T-02, T-03, T-04, T-05, T-55, T-57, T-70, T-72, T-77
C-02	<code>extract.py</code> (<code>SpecId</code> with title, text and recorded, <code>SpecIndex</code>)	T-01, T-06, T-72, T-77
C-03	<code>attribute.py</code> (<code>TestCase</code> , <code>Citation</code> , <code>test*</code> methods, Swift adapter), <code>extract.py</code> (exclusions incl. temporaries, binary, symlinks)	T-09, T-10, T-13, T-14, T-36, T-56, T-65, T-66, T-67
C-04	<code>results.py</code> (two-step <code>join_name</code>)	T-15, T-16, T-17, T-18, T-19, T-52, T-58, T-68
C-05	<code>graph.py</code> (<code>IdStatus</code> , <code>compute_status</code> , recorded skip in step 5)	T-20, T-21, T-27, T-53, T-77
C-06	<code>judge.py</code> (<code>JudgeRequest</code> with numbered source and full statement, <code>Verdict</code> with clause, validation incl. K-15), providers; <code>judge_mock.py</code> tokens	T-26, T-29, T-30, T-32, T-33, T-54, T-69, T-74, T-75
C-07	<code>report.py</code> (<code>to_json</code> ; title; recorded; clause; Decimal quantization; <code>verdict: null</code> ; Note order)	T-34, T-37, T-59, T-73, T-75, T-77
C-08	<code>report.py</code> (<code>to_markdown</code> ; Statement cell from title; (recorded) ID cell; §8 Clause column; em dash and (file) renderings)	T-35, T-73, T-75, T-77
C-09	<code>judge_llm.py</code> (<code>from_env</code>)	T-33, T-40
C-10	<code>speccheck/judge_prompt.md</code> , <code>judge_llm.py</code> (system message)	T-33, T-54, T-74, T-75
C-11	<code>judge.py</code> (progress line rendering, draw/erase sequences)	T-62
I-001	<code>report.py</code> (temp-and-rename, interrupt cleanup), <code>cli.py</code>	T-07, T-38, T-43, T-45, T-60, T-64
I-002	kernel modules	T-36
I-003	<code>report.py</code>	T-25, T-35
I-004	<code>graph.py</code>	T-27, T-28
I-005	<code>judge.py</code> (validation: evidence in span, clause located)	T-29, T-75
I-006	<code>cli.py</code> , module import layout	T-43
I-007	<code>cli.py</code> (logging filter), <code>judge_llm.py</code> (redaction)	T-40, T-41
I-008	<code>graph.py</code> (metrics)	T-24
I-009	<code>cli.py</code> , <code>report.py</code>	T-39
I-010	<code>graph.py</code> (edge selection: PASSING, passed, not recorded), <code>judge.py</code>	T-31, T-77
I-011	<code>extract.py</code> (normalization)	T-02
K-01	<code>cli.py</code>	T-39, T-40, T-19, T-64
K-02	<code>extract.py</code> (file filters, binary rule)	T-13
K-03	<code>extract.py</code> (walker, no symlinks)	T-13
K-04	<code>extract.py</code> (ID_RE)	T-03

Spec id	Where realized (component / module)	Verified by
K-05	judge_llm.py (full-lifecycle timeout)	T-33
K-06	judge_llm.py (concurrency), cli.py (--judge-concurrency)	T-33
K-07	judge.py	T-32
K-08	kernel (performance)	T-51
K-09	report.py	T-34
K-10	cli.py (--version)	T-50
K-11	cli.py (--max-unknown, Decimal comparison, null rule)	T-59
K-12	judge.py (budget deadline, in-flight completion), cli.py (--judge-budget)	T-61
K-13	judge.py (redraw on completion, 1 s ticker, single erase)	T-62
K-14	extract.py (statement cap at line boundary, marker line, Note)	T-72
K-15	judge.py (whitespace-collapsed substring matcher, 12/280 bounds)	T-75
E-01	extract.py, cli.py	T-07
E-02	extract.py	T-06
E-03	extract.py	T-04
E-04	extract.py (fence tracking)	T-05
E-05	results.py	T-19
E-06	results.py	T-17
E-07	results.py, report.py	T-18
E-08	graph.py (unrun)	T-22
E-09	cli.py (path validation incl. --out; symlinked roots resolved and used)	T-40
E-10	extract.py	T-13
E-11	extract.py (scan roots and SPEC.md)	T-13, T-01
E-12	attribute.py	T-11
E-13	attribute.py, report.py	T-10
E-14	judge.py, judge_llm.py	T-30, T-33
E-15	judge.py	T-30
E-16	judge.py	T-29
E-17	judge.py	T-26
E-18	report.py (write/rename/cleanup, JSON-then-Markdown)	T-45
E-19	graph.py, cli.py	T-20, T-39
E-20	extract.py (strikethrough only in spec)	T-23
E-21	cli.py	T-40
E-22	attribute.py	T-14
E-23	extract.py (exclusions)	T-36
E-24	results.py (join_name, worst-of)	T-52
E-25	graph.py (step 2b)	T-53
E-26	graph.py (step 5), cli.py (--strict)	T-27, T-39
E-27	results.py (longest-suffix join)	T-58
E-28	attribute.py (class recognition, test* methods)	T-56
E-29	extract.py (binary rule)	T-13
E-30	extract.py (walker; descent-time symlinks only)	T-13
E-31	extract.py (row/heading parsers)	T-55
E-32	cli.py (R-28 gate, unavailable precedence)	T-59
E-33	extract.py (ignore markers)	T-57
E-34	extract.py (temporary exclusion), report.py (leftover cleanup)	T-36, T-45
E-35	judge.py (budget)	T-61
E-36	judge.py, cli.py (R-28 with null)	T-59
E-37	graph.py (eligibility), report.py (verdict: null, em-dash rendering)	T-34, T-35, T-77
E-38	report.py (Note ordering)	T-34

Spec id	Where realized (component / module)	Verified by
E-39	<code>cli.py</code> (<code>--progress</code> gating)	T-63
E-40	<code>judge.py</code> (erase in <code>finally</code>)	T-63
E-41	<code>cli.py</code> (<code>KeyboardInterrupt</code> $\rightarrow$ exit 3), <code>report.py</code> (cleanup), <code>judge.py</code> (erase; timed polling of futures, <code>abort</code> flag), <code>judge_llm.py</code> (abortable deadline wait, daemon transport thread)	T-63 , T-64
E-42	<code>attribute.py</code> (Swift brace fallback)	T-67
E-43	<code>attribute.py</code> (Swift undelimited <code>test*</code>)	T-65 , T-66 , T-71
E-44	<code>extract.py</code> (first-cell decoration)	T-70 , T-71
E-45	<code>results.py</code> (tie $\rightarrow$ unattributed)	T-68
E-46	<code>extract.py</code> (cap and Note; empty body $\rightarrow$ title)	T-72
E-47	<code>extract.py</code> (inner declarations parsed; body ends only at level $\leq$ own)	T-72
E-48	<code>judge.py</code> (unlocated clause $\rightarrow$ UNKNOWN)	T-75
E-49	<code>judge.py</code> (clause blanked for UNRELATED / UNKNOWN, non-string clause, coerced verdicts)	T-75
E-50	<code>extract.py</code> (marker on a non-T id $\rightarrow$ Note)	T-77
E-51	<code>graph.py</code> (steps 1–4 unchanged for recorded ids)	T-77

12. Open questions and decisions to confirm

Every row below is a decision the specification’s author made on the requester’s behalf because `one_sentence_prompt.md` did not settle it. The normative rows assume the default; the reviews checked that the default is precise, not that it is what the requester wanted. Rows are the **D-*nn*** family (decisions to confirm). A row marked **confirm** is ratified by a human changing its status to **confirmed v1.n**; a row the requester overturns becomes a **fix(speccheck)**: change with a version bump.

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-01	Implementation stack	Python 3.12 + <code>uv</code> , standard-library kernel, one HTTP client behind an extra	Go or Rust (single static binary; stack-neutral repos would not need a Python runtime); TypeScript (nearest to most agent tooling)	§10, C-02/C-03 type pins, K-10	requester / confirm
D-02	Test-result input contract	JUnit XML, consumed; the tool never runs tests	run the suite itself (simpler for users, but executes untrusted code and ties the tool to one runner); pytest’s own JSON report (richer, Python-only)	R-05 , C-04 , non-goal “no test execution”	requester / confirm

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-03	What counts as a citation	a literal ID token anywhere in a file, with <code>speccheck:ignore</code> opt-outs	a structured annotation (<code>@spec</code> [R-07] (#R-07), a decorator, a registry) — precise but requires adoption; docstring-only citations — misses comments	C-01 , R-03 , R-04 , R-27 , the §0 limitation	requester / confirm
D-04	Family-T semantics	option (a): T ids are in scope, cited by their own test, and count in conformance	option (b): T ids declared but out of scope, reported separately — leaves conformance about R/C/I/K/E only	R-25 , C-05 step 2b, C-07 metrics, T-53	requester / confirm (F-001 chose (a) on the reviewer’s recommendation)
D-05	Mixed judge verdicts under <code>--strict</code>	an ID with both ASSERTS and EXECUTES_ONLY edges is PASSING and passes strict	a MIXED status that fails strict — stricter, noisier	R-15 , C-05 step 5, E-26	requester / confirm (F-003)
D-06	Judge question and vocabulary	one question per edge — does this test <i>assert</i> the behavior? — with ASSERTS / EXECUTES_ONLY / UNRELATED / UNKNOWN ; from v1.7 answered at the granularity of a clause: a statement with several clauses is asserted when any one of them is (C-10 , R-33)	a graded strength score (rejected: a number the model made up); judging code rather than tests (rejected: semantic analysis is a non-goal)	C-06 , C-10 , I-004 , I-005	requester / confirm

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-07	Judge request parameters	<code>temperature 0,</code> <code>max_tokens 4000</code> (v1.2; was 400), 30 s timeout, concurrency 4, budget unlimited, <code>--max-unknown</code> 0.2	any of these numbers; <code>max_tokens 400</code> was rejected after the first T-49 runs because thinking models (<code>qwen3:8b,</code> <code>gemma4:latest</code>) spend the budget on reasoning and are truncated before the verdict JSON, driving <code>unknown_rate</code> above <code>--max-unknown</code> ; at 4000 both pass T-49 three runs out of three. The other numbers remain defaults with no data behind them	C-06 , K-05 , K-06 , K-11 , K-12	requester / <code>max_tokens</code> confirmed; the rest remain confirm

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-08	Judge instruction text and model	the C-10 text as written (v1.9: clause-first question, clause field); no default model — the operator’s <code>SPECCKECK_JUDGE_MODEL</code>	any rewording; the text’s only evidence is T-49	C-10 , R-26 , T-49 , T-76	requester / confirm — re-opened again in v1.9: the text changed (R-34); confirm after three fresh T-49 runs over the T-76 label set with each candidate model, and record both. Measured 2026-09-18 under the v1.8 text: <code>openai/gpt-4o-mini</code> (fast; graded long bodies by their gist) and <code>google/gemini-3.8-flash</code> (located clauses, stricter, 4–8× the wall-clock, needs <code>--judge-concurrency 8</code>) — both passed the nine-label T-49 at 1.000, which is why T-76 exists
D-09	Input limits	files over 2 MiB skipped; NUL byte in the first 8 KiB means binary; IDs 1–3 digits	larger cap, or none; MIME sniffing; 4-digit IDs	K-02 , K-04 , E-10 , E-29	requester / confirm
D-10	LLM endpoint shape	an OpenAI-compatible <code>chat/completions</code> body, provider-agnostic	a specific vendor SDK (simpler, vendor-locked); a local-model-only path	C-06 , C-09	requester / confirm
D-11	Fixture and self-check layout	<code>fixtures/target/</code> with <code>golden/</code> , byte-copied into the wheel as <code>speccheck/_selfcheck</code>	package only the goldens and regenerate the fixture; skip the self-check entirely	§10 , T-43 , T-46 , T-60	requester / confirm

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-12	Report file names and location	<code>SPEC_CONFORMANCE_REPORTS</code> directory; and <code>speccheck.json</code> under <code>--out</code> , default <code>.</code>	<code>REPORTS</code> directory; a single JSON with Markdown derived by a separate renderer	§3.3, C-07, C-08, E-18	requester / confirm
D-13	Python test-case delimitation	<code>def test_*</code> at module level; <code>test*</code> methods in <code>Test*</code> classes and <code>*TestCase</code> subclasses; nested classes excluded	honor <code>pytest.ini</code> <code>python_functions</code> / <code>python_classes</code> ; collect via <code>pytest</code> itself (rejected: runs code)	C-03, E-28, T-56	requester / confirm (F-007, F-102)
D-14	Reference machine for K-08	named in <code>SPEC_BUILD_REPORT.md</code> at build time	a CI runner with a generous bound; no performance constraint at all	K-08, T-51	build owner / open
D-15	Progress indicator shape and gating	a single in-place ASCII line on <code>stderr</code> (<code>#/-</code> bar of 20 cells, done/total, elapsed, ETA per C-11), padded with spaces and redrawn with a bare <code>\r</code> (no terminal escapes; v1.4, F-205), on by default only when <code>stderr</code> is a TTY and not at <code>DEBUG</code> , erased when the judge stage ends, <code>--progress</code> <code>auto\ always\ never</code> to override; LLM judge only	leaving the final line on screen (rejected: §5.3 quiet-by-default would then have a visible exception at exit); a per-edge log line instead of a bar (rejected: that is what <code>--verbose DEBUG</code> already is); Unicode block characters (rejected: R-29 locale reasoning); a bar for <code>--judge mock</code> too (rejected: mock is sub-second, K-08); a spinner without ETA (rejected: the requester's complaint is duration, so ETA is the useful number)	R-30, C-11, K-13, E-39, E-40, §5.1, §5.3	requester / confirm (the <i>existence</i> of the bar is the requester's ask of 2026-09-13; its shape is the author's default)
D-16	Exit code on interrupt	<code>SIGINT/KeyboardInterrupt</code> exits 3 with message <code>interrupted</code> , keeping K-01's closed set <code>{0,1,2,3}</code> ; temporaries and any already-renamed report removed	the shell convention 130 (rejected by default: it widens K-01 and every CI wrapper that switches on the code; easy to adopt if the requester prefers it); leaving Python's default (rejected: traceback, exit 1, indistinguishable from NOT CONFORMING)	E-41, E-40, §5.4, K-01, I-001, T-64	requester / confirm (F-204)

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-17	How Swift test files are delimited	by lines and brace counting inside <code>attribute.py</code> (C-03 TYPE LINE / FUNC LINE / ATTRIBUTE BLOCK / SPAN END rules; comments and string literals excluded from the count), so the kernel stays standard-library and needs no toolchain at check time	a real parser (<code>swift-syntax</code> has no Python binding; <code>swiftc -dump-parse</code> or <code>swift test list</code> need a Swift toolchain where the checker runs and give names but not line spans); treating <code>.swift</code> as file-level as before (the citations then never join — the defect that motivated v1.6)	R-31, C-03, E-42, E-43, T-65..T-67	requester / confirm
D-18	Where the Swift <code>MODULE</code> in a classname comes from	the first path component of the file under its <code>--tests</code> root, or the root's last component for a file directly under it (SwiftPM's <code>Tests/<Target>/</code> layout)	parse <code>Package.swift</code> for target names and paths (a second grammar); a <code>--swift-module</code> flag (one more thing to get wrong); match on the type chain alone ignoring the module (ambiguous across targets)	C-03, T-65, T-68	requester / confirm
D-19	How a Swift Testing result name (<code>twoArgs(a:b:)</code>) is joined	C-04 strips the signature and joins on the bare identifier; overloads by label tie and are unattributed (E-45)	reconstruct the label signature in the adapter and join exactly (correct for overloads, but default arguments, <code>_</code> labels, generics and <code>inout</code> all need parsing to get right); join on signature when present and fall back to identifier (two rules where one suffices)	C-04, E-45, T-68	requester / confirm

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-20	What a heading-declared ID's statement contains	the title plus the whole section body, fenced code blocks included, capped at 16,384 bytes (R-33 , C-01 (b), K-14) — the pinned shape is the contract, and it is what a literal judge needs	prose only, code blocks dropped (smaller; rejected because MonteCarloPi's C-02 would still be judged on comment-free prose); the first fenced block only (loses prose clauses such as “throws on the first tick”, and bullet-pinned shapes such as C-05 's reservoir); the body to the judge but the title as JSON statement (two notions of “statement” in one tool; the report could not show what the judge saw); letting the judge fetch context itself (nondeterministic, provider-specific, against the §0 boundary). The cap's value is part of this decision: the proposal drafted 8,192 bytes, which would have truncated this document's own C-03 (8.4 kB) in T-48 and handed the judge a Swift-less C-03 ; the requester raised it to 16,384 on 2026-09-18 so no known contract is cut	R-33 , C-01 , C-02 , C-06 , C-07 , C-10 , K-14 , E-46 , E-47 , T-72 .. T-74	requester / confirmed v1.7 (code blocks included, cap 16,384; 2026-09-18)

ID	Decision	Default taken	Alternatives rejected	Affects	Owner / status
D-21	How a verdict is tied to the statement	the judge quotes the clause it judged against and the kernel requires it to be LOCATED (whitespace-collapsed substring, 12..280 characters; K-15), coercing an unlocated ASSERTS/EXECUTES_ONLY to UNKNOWN — the same grounding pattern as evidence lines (R-34 , E-48)	prompt rewording only, no <code>clause</code> field (smaller; unverifiable — the kernel cannot tell a located match from a gist, and the report cannot show which clause was judged); sending only a “relevant” slice of the body (the kernel cannot know which clause a test is about; D-20 ’s rejected alternative); voting across repeated calls (K-06 : one request per edge; hides instability); fuzzy clause matching (arguable at the boundary; “quote it” means verbatim, and a model that cannot is the finding)	R-34 , C-06 , C-07 , C-08 , C-10 , K-15 , E-48 , E-49 , I-005 , T-75	requester / confirmed v1.9 (2026-09-18: “apply proposal 1.9 in full”)
D-22	Recorded-not-gating T ids under an honest judge	v1.10: a <code>*(recorded)*</code> marker in the T row’s ID cell (C-01) makes the id RECORDED — its citing test still has to exist and pass (C-05 steps 1–4, E-51), but its edges are never judged (step 5, I-010), so a presence check is what it is and the strict LLM gate stays about assertions. This document marks T-48 , T-49 and T-51	no rule (v1.9’s state: an honest judge marks the presence check EXECUTES_ONLY and <code>--strict --judge</code> llm exits 1 on this repository by design — <code>gemini-3.8-flash</code> did so to T-48 on 2026-09-18); a stated assertion for the presence check (fragile: what a presence check can honestly assert differs per recorded test); excluding family T from the judge stage entirely (too broad: T ids realized by real tests benefit from the judge)	R-35 , C-01 , C-02 , C-05 step 5, I-010 , C-07 , C-08 , E-50 , E-51 , T-77	requester / confirmed v1.10 (2026-09-18: “let’s do D-22 ”)

None of the first fourteen was raised as a question before v1.1; each was decided and reviewed for precision only. That is the defect this section corrects: a specification can be implementation-grade and still not be what was asked for.

Revision history

Version	Change
v0.1	Initial draft from <code>one_sentence_prompt.md</code> , for <code>spec-review</code> .
v0.2	P0 of <code>SPEC_REVIEW_REPORT.md</code> applied: F-001 T-family semantics (C-01, C-05 step 2b, R-25, E-25, T-53, T-48); F-002 scan exclusions and <code>--out</code> inside <code>--root</code> (C-03, §5.1, E-09, E-23, T-36); F-003 status is the sole input to <code>--strict</code> (R-15, §5.4, C-05, E-26, T-27, T-39); F-004 LLM body, response path, and normative instruction text (C-06, C-10, R-26, C-07 <code>judge_prompt_sha256</code> , T-33, T-54); F-005 parametrized JUnit names (C-04, C-07 <code>results</code> , E-24, T-52). P1 (F-006..F-013) and P2 (F-014..F-017) remain open.
v0.3	P1 of <code>SPEC_REVIEW_REPORT.md</code> applied: F-006 longest-suffix classname join (C-04, E-27, T-58); F-007 <code>*TestCase</code> classes and <code>async def</code> (C-03, E-28, T-56); F-008 binary rule and no symlinks (C-03, K-02, K-03, E-29, E-30, T-13); F-009 row/cell/heading declaration grammar (C-01, E-31, T-55); F-010 one-line ASCII summary with regex, UTF-8 stdout (§5.1, R-29, T-44); F-011 packaged self-check fixture in a temp dir, I-001 exemption, §9.8 reference (§5.1, I-001, §10, T-43); F-012 <code>--strict --judge llm</code> requires an available judge and <code>unknown_rate ≤ --max-unknown</code> (R-28, K-11, C-07, §5.4, E-32, T-59); F-013 <code>speccheck:ignore</code> markers and the stated literal-citation limitation (§0, C-01, R-27, E-33, T-57). P2 (F-014..F-017) remains open.
v0.4	P2 of <code>SPEC_REVIEW_REPORT.md</code> applied: F-014 logging default <code>ERROR</code> , Notes at <code>INFO</code> , nothing at <code>WARNING</code> (§5.3, T-42); F-015 in-memory render, temp-and-rename dual write, cleanup on failure, <code>--out</code> creation permitted (§3.1, §3.3, I-001, E-18, T-45); F-016 <code>judge_available</code> null/true/false rule, <code>by_family</code> always six keys, <code>by_status</code> seven in-scope statuses, ID cell struck (C-07, C-08, T-34, T-35); F-017 R-24 scoped to deterministic statuses plus recorded verdicts, T-49 per-run not pooled, K-08/T-51 recorded on a named reference machine via <code>tools/bench.py</code> , <code>SPEC.md</code> decoded UTF-8-with-replacement (R-24, T-49, K-08, T-51, §5.1, E-11, T-01). No findings open.
v0.5	<code>FINAL_SPEC_REVIEW_REPORT.md</code> P1 and P2 applied: F-101 <code>join_name</code> strips from the first [, T-52 corrected; F-102 <code>test*</code> methods (no underscore) in recognized classes (C-03, E-28, T-56); F-103 temporaries get a per-run nonce, are excluded from scans, and leftovers are deleted first (§3.1, C-03, E-34, T-36, T-45); F-104 R-10 passed-outcome clause, R-19 self-check exemption; F-105 empty <code>classname</code> join and <code>null unknown_rate</code> (C-04, K-11, E-36, T-52, T-59); F-106 JSON-then-Markdown rename order and the one permitted stale combination (§3.1, E-18, T-45); F-107 inner self-check exit expectation, <code>_selfcheck/</code> byte-identical to <code>fixtures/target/</code> , drift guard T-60 (§5.1, §10); F-108 fence definition and fallback <code>classname</code> (C-01, C-03, T-05); F-109 line-numbered <code>source</code> in the judge request, marker fixtures in data files (C-06, C-10, §9); F-110 <code>--judge-concurrency</code> and <code>--judge-budget</code> (K-06, K-12, §5.1, E-35, T-33, T-61). No findings open.
v1.0	All eleven findings of the independent <code>SPEC_v0.5_REVIEW_REPORT_by_QWEN.md</code> applied (cited as Q-nnn): Q-001 <code>verdict</code> key always present, <code>null</code> when unjudged, em dash in Markdown (C-07, C-08, E-37, T-34, T-35); Q-002 <code>notes</code> sorted by code point (C-07, E-38, T-34); Q-003 exact in-process <code>--self-check</code> invocation with <code>--results</code> , goldens under <code>fixtures/target/golden/</code> (§5.1, §10, T-43, T-46, T-60); Q-004 <code>unavailable</code> takes precedence over <code>unknown_rate</code> (C-07, §5.1, E-32, T-59); Q-005 <code>skipped</code> count added to the summary line and regex (§5.1, T-44); Q-006 “first declaration wins” removed (C-01); Q-007 symlinked command-line roots resolved and descended, descent-time symlinks skipped (C-03, E-09, E-30); Q-008 fence closing pinned — same marker character, no info string, no cross-type closure (C-01, T-05); Q-009 ratios as quantized Decimals emitted with exactly four decimals, K-11 compares the quantized value (C-07, K-11, T-34); Q-010 K-05 timeout spans the full request lifecycle, K-12 deadline semantics with in-flight completion (K-05, K-12, E-35, T-61); Q-011 <code>name</code> is <code>""</code> in JSON, (file) is Markdown-only, R-13 concerns data (C-07, C-08, E-13, T-35). No findings open.

Version	Change
v1.1	Added §12, <i>Open questions and decisions to confirm</i> : fourteen decisions the author took by default on the requester’s behalf (stack, results contract, citation rule, T-family option, mixed-verdict strict rule, judge question and parameters, instruction text, input limits, endpoint shape, fixture layout, report names, test delimitation, reference machine), each with its default, the alternatives rejected, the IDs it affects, and a confirm / open status. No normative row changed. The spec-writing skill now requires this section.
v1.2	D-07 partly resolved by the first T-49 runs: max_tokens raised from 400 to 4000 in the judge request body (C-06), the wire-format test updated to match (T-33), and the D-07 row records the evidence. Thinking models truncated at 400 before emitting the verdict JSON; at 4000 qwen3:8b and gemma4:latest pass T-49 three runs out of three. No other normative row changed.
v1.3	Judge-stage progress indicator, requested on 2026-09-13 because <code>--judge llm</code> runs take minutes: R-30 (requirement), C-11 (the one-line ASCII format, draw/erase byte sequences, regex, <i>k</i> and ETA formulas), K-13 (redraw cadence, 20 cells, single erase before the report stage), E-39 (no bytes when not a TTY, at DEBUG, under <code>--progress never</code> , or with <code>--judge none\ mock</code>), E-40 (erase on interrupt or failure), <code>--progress auto\ always\ never</code> in §5.1, the §5.3 quiet-by-default exception, T-62/T-63 , §11 rows, D-15 . The indicator is written to the raw stderr stream, not the logger, and is erased at the end of the stage, so stdout, both reports, the exit code, and what remains on stderr at exit are unchanged.
v1.4	All ten findings of the v1.3 SPEC_REVIEW_REPORT.md applied. P1: F-201 the logger is silent while the indicator is displayed (C-11 , K-13 , §5.3, T-62); F-202 one clock origin t_0 = first draw, K-12 keeps its own (C-11 , K-13 , T-62); F-203 coalesced cadence — within 100 ms, $\geq 1/s$ in flight, $\leq 10/s$ — and atomic serialized writes (K-13 , T-62); F-204 E-41 interrupt rule: exit 3, message interrupted , temporaries and renamed reports removed (E-40 , E-41 , §3.1, §5.4, T-63 , T-64 , §11, D-16); F-206 C-07 example uses 0.2000/0.0000 and max_unknown is emitted quantized (C-07). P2: F-205 no terminal escapes — draws padded to the widest line so far, erase is <code>\r + spaces + \r</code> (C-11 , T-62 , D-15); F-207 C-08 header parenthetical omitted when judge_available is null (C-08 , T-35); F-208 T-46 cited by R-01..R-15 and T-60 by R-18/I-001 in §11; F-209 I-005 vacuous clause replaced; F-210 T-49 names <code>fixtures/target/golden/judge_labels.json</code> and <code>tools/eval_judge.py</code> .
v1.5	E-41 tightened from “in-flight requests are not awaited beyond the K-05 timeout” to “abandoned; exit within 1 s” after the first interrupt of a real <code>--judge llm</code> run against a local model appeared to be ignored: the main thread was blocked in an untimed future wait (not SIGINT-interruptible on macOS CPython) and then waited for the in-flight requests. T-64 gains the timing case; §11 E-41 row names the mechanism. No other row changed.
v1.6	Swift adapter and two small grammar changes, requested 2026-09-17 after the MonteCarloPi Swift build ran the checker and got 62 unverified ; 43 dangling purely from tool limits: R-31 Swift Testing / XCTest test-case delimiting by lines with doc-comment-inclusive spans and SwiftPM-shaped classnames (C-03 , E-42 , E-43 , D-17 , D-18 , T-65..T-67), a second join_name step that strips a Swift signature (C-04 , E-45 , D-19 , T-68), Swift assertion tokens for the mock judge (C-06 , T-69), R-32 decoration after the bold ID in a declaring first cell (C-01 , E-44 , T-70), a Swift golden fixture (T-71), O-2 and the §0 non-goal updated.

Version	Change
v1.7	A heading-declared ID's statement is its title plus its section body (PROPOSAL_v1.7_heading_bodies.md, 2026-09-17): two LLM judges over the same 130 MonteCarloPi edges disagreed almost entirely on <code>###</code> -declared contracts because both were handed a title (<code>Data structures</code> , <code>`EstimationWorker` (an `actor`)</code>) and never the pinned API beneath it — one guessed generously, the other refused, and neither had the contract. R-33 ; C-01 (b) HEADING LINE / SECTION BODY grammar, fenced blocks included (D-20), table declarations unchanged; C-02 <code>SpecId.title</code> ; C-06 statement passthrough; C-07 title key and <code>schema_version "1.1"</code> ; C-08 renders title, so the Markdown report is byte-identical; C-10 gains the any-clause rule (new <code>judge_prompt_sha256</code>); K-14 16,384-byte cap with marker line and Note (the proposal's 8,192 would have truncated this document's C-03); E-46 , E-47 ; T-72..T-74 ; §11 rows; D-20 . The mock judge, statuses, metrics, and exit codes are unchanged; <code>--judge none</code> output differs only in <code>speccheck.json</code> .
v1.8	All seven findings of the v1.7 SPEC_REVIEW_REPORT.md applied. P1: F-301 line model for SPEC.md — split on <code>\n</code> , trailing <code>\r</code> removed, <code>BLANK := empty or whitespace-only</code> , body lines re-joined with <code>\n</code> (C-01 , C-02 , I-002 , T-72); F-302 T-72 and T-73 restated as properties of the current output, the one-time golden diff moved to SPEC_BUILD_REPORT.md; F-303 T-49 counts only runs under the current <code>judge_prompt_sha256</code> , D-08 re-opened, D-06 and §0 name the clause granularity. P2: F-304 ATX headings only, at most three leading spaces, bare <code>###</code> is a heading with an empty title, trailing <code>#</code> run stripped from the title, the declaring line is a HEADING LINE (C-01 , T-72); F-305 empty title with a body → statement is the body alone (C-01 , C-02 , E-46 , T-72); F-306 C-09 cited by T-33 and T-40 ; F-307 revision table sorted ascending, Status bullet shortened, title in the §1 Extractor row. No behaviour the v1.7 build would produce differently was changed; what changed is what the spec pins.
v1.9	Clause-grounded verdicts and a body in the judge-evaluation fixture (PROPOSAL_v1.9_clause_grounding.md, 2026-09-18; D-21 confirmed). Evidence: with bodies in the statement, <code>gpt-4o-mini</code> graded long contracts by their gist — 13 of 18 stable downgrades on the mdv tree were tests asserting a body clause nearly verbatim — while <code>gemini-3.8-flash</code> located the clauses and found real gaps (<code>mdv: [E-19] (#E-19)</code> , <code>mdv: [T-34] (#T-34)</code> , this document's R-10); both passed the nine-label T-49 at 1.000. R-34 ; C-06 <code>Verdict.clause</code> , K-15 LOCATED rule, E-48 (<code>judge: unlocated clause</code>), E-49 (clause blanked for UNRELATED/UNKNOWN), I-005 grounded on both sides, mock clause = prefix of the statement (R-22); C-07 clause key and <code>schema_version "1.2"</code> ; C-08 §8 Clause column; C-10 question reworded clause-first, <code>clause</code> in the reply (new <code>judge_prompt_sha256</code>); T-75 ; golden fixture gains a heading-declared contract with a multi-clause body 2,048 bytes and six labeled tests, label set 20 entries (T-46 , T-76 , T-49); §11 rows; D-08 re-opened and made a model row; D-21 ; D-22 opened (recorded-not-gating T ids under an honest judge). Statuses, metrics, exit codes, and <code>--judge none/mock</code> Markdown output unchanged; <code>--judge mock</code> JSON gains the <code>clause</code> key.
v1.10	D-22 settled: RECORDED tests. A T row whose ID cell carries <code>*(recorded)*</code> as the first decoration token declares a recorded id (C-01 , C-02 <code>recorded</code>); its status is computed by C-05 steps 1–4 like any T id (E-51) but its edges are never judged (step 5, I-010) and are not eligible edges for C-11 ; the marker on a non-T id is ignored with a Note (E-50); JSON gains <code>recorded</code> per id (<code>schema_version "1.3"</code>), the Markdown ID cell reads <code>[T-48] (#T-48) (recorded)</code> (C-07 , C-08); R-35 , T-77 ; T-48 , T-49 and T-51 marked in this document. Motivation: under an honest judge the presence checks that keep those ids cited are EXECUTES_ONLY by construction, so <code>--strict --judge llm</code> was red on this repository for a reason the spec intended but had left implicit.

Version	Change
v1.11	All seven findings of the v1.10 SPEC_REVIEW_REPORT.md applied. P0: F-401 <code>schema_version</code> stated once (C-07, §3.3), T-73/T-75/T-77 refer to “the C-07 value”. P1: F-402 the C-06 validation list is numbered, first-match, with a non-string <code>clause</code> treated as absent and every coerced UNKNOWN recording <code>clause ""</code> (C-06, E-16, E-49, T-75); F-403 RECORDED named in the null-verdict condition, E-37 owning it (E-37, C-07, T-34); F-404 one marker rule after C-01 (b), the heading title strips it, ~T-48~ (recorded) pinned (C-01, C-08, T-77); F-405 <code>judge_strength</code> over PASSING RECORDED, null case stated, T-77 arithmetic (C-07). P2: F-406 K-15 trims the clause (T-75 sub-case); F-407 “the current C-10 text”, D-08 measurement moved to its status cell, Status bullet trimmed. No behaviour the build would otherwise get wrong changed; what changed is what the spec pins.